UNIVERSIDAD DEL QUINDÍO
Facultad de Ingeniería · Ingeniería de Sistemas y Computación

Programación III

PARCIAL 2

Archivos, Streams, structs y recursión en Elixir

Problemas elegidos: 1. Rutas entre ciudades · 2. Escapar de la isla

Presentado por:
César Leandro Ortegón

Docente:
Carlos Andrés Flórez Villarraga

Repositorio: https://github.com/leandro-ortegon/PARCIAL-2
Armenia, noviembre de 2026

Versión final: Problemas 1 y 2 completos

Tabla de contenido

1. Introducción y organización del trabajo 5
1.1 Problemas elegidos 5
1.2 Modalidad de trabajo 5
1.3 Repositorio y estructura 5
1.4 Cómo ejecutar 5
1.5 Requisitos generales y dónde se cumplen 6
2. Problema 1: Rutas entre ciudades 7
2.1 Enunciado resumido 7
2.2 Arquitectura 7
2.3 Datos de entrada 8
2.4 Desarrollo por módulo 9
2.4.1 Conexion (conexion.exs) 9
2.4.2 Lector (lector.exs) 10
2.4.3 Depuracion (depuracion.exs) 12
2.4.4 Grafo (grafo.exs) 14
2.4.5 Alcance (alcance.exs) 15
2.4.6 Ruta (ruta.exs) 16
2.4.7 Busqueda (busqueda.exs) 17
2.4.8 Reporte (reporte.exs) 18
2.4.9 Salida (salida.exs) 19
2.4.10 Programa (programa.exs) 19
2.5 Validación con recursión: depuración de conexiones 21
2.5.1 Casos base y casos recursivos 21
2.5.2 Prueba de escritorio 21
2.5.3 Árbol de llamadas recursivas 22
2.5.4 Resultado obtenido 22
2.6 Streams infinitos: ciudades alcanzables 22
2.6.1 Funcionamiento 22
2.6.2 Prueba de escritorio (origen: Armenia) 23
2.6.3 Resultado obtenido 23
2.7 Búsqueda principal: todas las rutas 24
2.7.1 Funcionamiento 24
2.7.2 Árbol de llamadas: Armenia → Manizales 24
2.7.3 Prueba de escritorio 25

2.8 Resultados obtenidos..... 25
2.9 Casos de error..... 28
2.10 Estado de validación del problema 1..... 28
2.11 Bitácora de uso de IA (problema 1)..... 28
2.11.1 Interacciones registradas..... 29
2.11.2 Un ejemplo concreto..... 30
3. Problema 2: Escapar de la isla..... 31
3.1 Enunciado resumido..... 31
3.2 Arquitectura..... 31
3.3 Datos de entrada..... 32
3.4 Desarrollo por módulo..... 33
3.4.1 Terreno (terreno.exs)..... 33
3.4.2 Ruta (ruta.exs)..... 34
3.4.3 Lector (lector.exs)..... 35
3.4.4 Validacion (validacion.exs)..... 37
3.4.5 Mapa (mapa.exs)..... 40
3.4.6 Busqueda (busqueda.exs)..... 42
3.4.7 Reporte, Salida y Programa..... 43
3.5 Validación con recursión: el mapa de la isla..... 46
3.5.1 Funcionamiento..... 46
3.5.2 Prueba de escritorio con isla_invalida.txt..... 47
3.5.3 Árbol de llamadas recursivas (fila 3 del mapa inválido)..... 47
3.6 Búsqueda principal: todas las rutas..... 48
3.6.1 Funcionamiento..... 48
3.6.2 Árbol de llamadas (primeros niveles)..... 48
3.6.3 Prueba de escritorio: ruta de menor costo..... 49
3.6.4 Rutas encontradas..... 49
3.7 Resultados del programa completo..... 49
3.7.1 Mapa del enunciado..... 50
3.7.2 Mapa inválido..... 51
3.7.3 Archivo inexistente..... 51
3.8 Casos de error..... 52
3.9 Estado de validación del problema 2..... 52
3.10 Bitácora de uso de IA (problema 2)..... 52
3.10.1 Interacciones registradas..... 52

3.10.2 Un ejemplo concreto..... 53
4. Reflexión sobre el uso de inteligencia artificial..... 54
4.1 Herramientas y agentes utilizados..... 54
4.2 Propuestas distintas a lo visto en clase..... 54
4.3 Conclusiones..... 54
Anexo A. Control de evidencias..... 56

1. Introducción y organización del trabajo

1.1 Problemas elegidos

El parcial exige resolver 2 de 4 problemas, con la condición de que al menos uno sea el 1 o el 4 (Streams infinitos) y al menos uno sea el 1 o el 2 (structs). Se eligió la combinación **1 y 2**, que es válida: el problema 1 cubre los Streams infinitos y ambos usan structs.

Se escogieron porque comparten la misma idea central, la búsqueda recursiva de todos los caminos sin repetir posiciones (backtracking), el struct `%Ruta{}` y el manejo de líneas inválidas en los archivos de entrada.

1.2 Modalidad de trabajo

El parcial se desarrolló de forma individual. El repositorio registra el avance por etapas, con un commit por cada módulo terminado y probado.

1.3 Repositorio y estructura

Repositorio: **https://github.com/leandro-ortegon/PARCIAL-2**

```
PARCIAL-2/
├── README.md
├── .gitignore
├── Problema1_Rutas/
│   ├── entradas/        conexiones.csv, consultas.txt
│   ├── salidas/         (la crea el programa)
│   ├── conexion.exs     struct Conexion y parse/1
│   ├── lector.exs       lectura perezosa con File.stream!
│   ├── depuracion.exs   validación recursiva (sin Enum ni Stream)
│   ├── grafo.exs        mapa ciudad => vecinos
│   ├── alcance.exs      Stream infinito de ciudades alcanzables
│   ├── ruta.exs         struct Ruta
│   ├── busqueda.exs     búsqueda recursiva con backtracking
│   ├── reporte.exs      construcción de líneas de texto (puro)
│   ├── salida.exs       escritura línea por línea
│   └── programa.exs     main
└── Problema2_Isla/      (en desarrollo)
```

Figura 1. *Estructura de carpetas del repositorio.*

1.4 Cómo ejecutar

Requisito: Elixir instalado (se usó Elixir 1.19.5 sobre Erlang/OTP 28). Desde la carpeta de cada problema:

```
cd Problema1_Rutas
elixir -r conexion.exs -r lector.exs -r depuracion.exs -r grafo.exs -r alcance.exs -r
ruta.exs -r busqueda.exs -r reporte.exs -r salida.exs programa.exs
```

Cada `-r` carga un módulo de apoyo; el orden importa porque `busqueda.exs` usa el struct definido en `ruta.exs`. Los resultados quedan en la carpeta `salidas/` y también se muestran en pantalla.

1.5 Requisitos generales y dónde se cumplen

Requisito del parcial	Cómo se cumple (Problema 1)
1. Lectura con `File.stream!/1`	`Lector.leer/2`: único punto donde se abren archivos; nunca se usa `File.read!`
2. Transformaciones perezosas	`Stream.with_index`, `Stream.map` y `Stream.reject`; un solo `Enum.reduce` al final
3. Ignorar vacías y comentarios	`Lector.ignorar?/1`
4. Escritura línea por línea	`Salida.escribir_lineas/2` con `Enum.into(File.stream!(ruta, [:write]))`
5. Pantalla y archivo	`Programa.publicar/2` escribe el archivo y muestra las mismas líneas
6. Archivo inexistente sin excepción	`File.exists?` antes de `File.stream!` y `with` en `main`
7. Pruebas de escritorio y árboles	Secciones 2.5, 2.6 y 2.7
Validación con recursión propia	`Depuracion`: sin `Enum` ni `Stream`

Tabla 1. *Requisitos generales del parcial y su implementación.*

2. Problema 1: Rutas entre ciudades

2.1 Enunciado resumido

Dado un conjunto de ciudades conectadas por rutas, el programa debe encontrar, para cada consulta origen-destino, todas las rutas posibles sin repetir ciudades, indicando para cada una el número de ciudades y la distancia total. Además debe informar cuántas rutas existen, cuáles son las más cortas (menos ciudades) y cuál tiene la menor distancia.

Componentes adicionales exigidos: lectura perezosa con Streams, structs `%Conexion{}` y `%Ruta{}`, reporte de líneas mal formadas, depuración recursiva de conexiones y un Stream infinito que determina si el destino es alcanzable antes de buscar rutas.

2.2 Arquitectura

Módulo	Responsabilidad	¿Puro?
`Conexion`	Struct y conversión de una línea en conexión	Sí
`Lector`	Lectura perezosa de los archivos de entrada	No (lee disco)
`Depuracion`	Validación recursiva de las conexiones	Sí
`Grafo`	Mapa de vecinos en ambos sentidos	Sí
`Alcance`	Stream infinito de ciudades alcanzables	Sí
`Ruta`	Struct de una ruta y su texto	Sí
`Busqueda`	Todas las rutas con backtracking	Sí
`Reporte`	Líneas de texto de cada archivo	Sí
`Salida`	Escritura línea por línea y pantalla	No
`Programa`	`main`: orquesta el flujo	No

Tabla 2. *Módulos del problema 1. La lógica es pura y los efectos (disco y pantalla) quedan en Lector, Salida y Programa.*

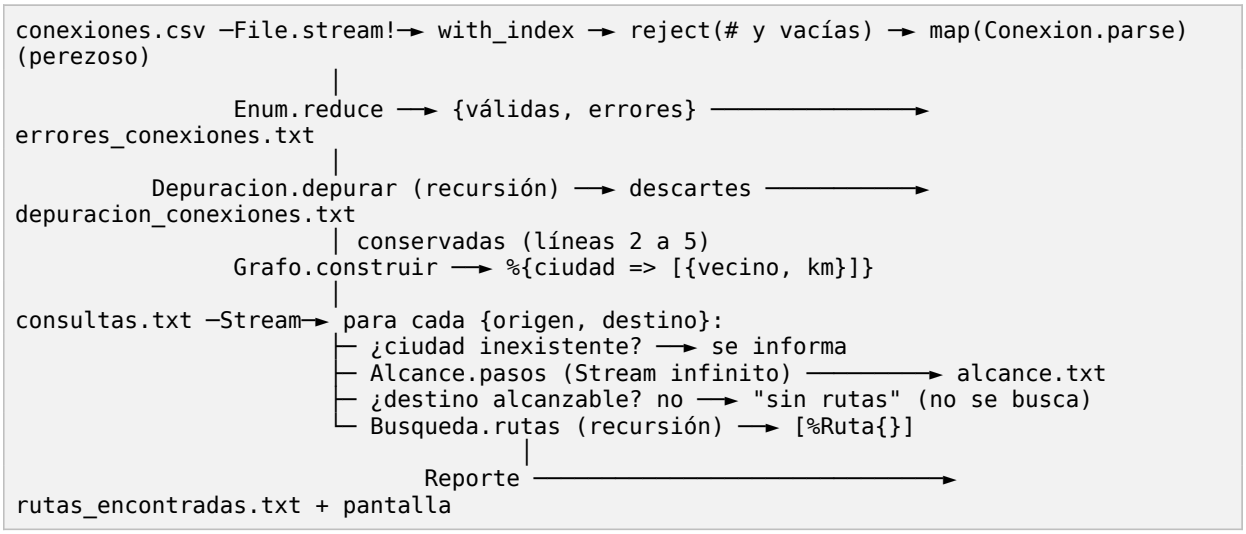


Figura 2. *Recorrido de la información en el problema 1.*

2.3 Datos de entrada

Se usaron los datos del ejemplo del parcial (líneas 1 a 8 de `conexiones.csv` y 1 a 2 de `consultas.txt`), más líneas de prueba agregadas al final para verificar los casos de error.

```
 1  # ciudad_a,ciudad_b,distancia_km
 2  Armenia,Pereira,45
 3  Pereira,Manizales,52
 4  Armenia,Cartago,60
 5  Cartago,Pereira,25
 6  Pereira,Armenia,45
 7  Manizales,Manizales,0
 8  Cartago,Pereira,30
 9
10  # lineas de prueba: errores de formato y una zona aislada
11  Armenia,Calarca
12  Pereira,Salento,abc
13  Bogota,Tunja,140
```

Figura 3. *Archivo entradas/conexiones.csv con números de línea.*

Línea	Contenido	Propósito de la prueba
1, 10	Comentarios	Deben ignorarse (requisito 3)
9	Línea vacía	Debe ignorarse (requisito 3)
2 a 5	Conexiones válidas	Forman el grafo depurado
6	`Pereira,Armenia,45`	Repetida en sentido contrario de la línea 2
7	`Manizales,Manizales,0`	Ciudad consigo misma
8	`Cartago,Pereira,30`	Mismo par que la línea 5 con otra distancia
11	`Armenia,Calarca`	Campos faltantes
12	`Pereira,Salento,abc`	Distancia no numérica
13	`Bogota,Tunja,140`	Conexión válida pero aislada: prueba de "sin rutas"

Tabla 3. *Propósito de cada línea de conexiones.csv.*

```
1  Armenia,Manizales
2  Cartago,Manizales
3  # consultas de prueba
4  Armenia,Bogota
5  Armenia,Cali
6  Pereira
```

Figura 4. *Archivo entradas/consultas.txt con números de línea.*

Las consultas 1 y 2 son las del parcial. La 4 prueba un destino existente pero inalcanzable, la 5 una ciudad que no existe y la 6 una consulta mal formada.

2.4 Desarrollo por módulo

2.4.1 Conexion (conexion.exs)

Define el struct `%Conexion{origen, destino, distancia}` con los tres campos obligatorios (`@enforce_keys`), de modo que nunca puede existir una conexión incompleta.

`parse/1` convierte una línea de texto en `{:ok, %Conexion{}}` o `{:error, motivo}`. Primero limpia la línea (`String.trim`), la separa por comas (`String.split`) y limpia cada campo (`Enum.map(&String.trim/1)`). Con pattern matching verifica que haya exactamente tres campos; si no, devuelve "campos faltantes".

`convertir_distancia/3` usa `Integer.parse/1` y solo acepta el resultado `{numero, ""}`, es decir, un entero completo. Así, "abc" y "4.5" se rechazan como "distancia no numerica". Es la misma técnica que el docente usa en `Util.leer/2`.

```
defmodule Conexion do
  @moduledoc """
  Representa una conexión entre dos ciudades y su distancia en kilómetros.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  @enforce_keys [:origen, :destino, :distancia]
  defstruct [:origen, :destino, :distancia]

  def parse(linea) do
    campos =
      linea
      |> String.trim()
      |> String.split(",")
      |> Enum.map(&String.trim/1)

    case campos do
      [origen, destino, distancia] ->
        convertir_distancia(origen, destino, distancia)

      _ ->
        {:error, "campos faltantes"}
    end
  end

  defp convertir_distancia(origen, destino, texto) do
    case Integer.parse(texto) do
      {distancia, ""} ->
        {:ok, %Conexion{origen: origen, destino: destino, distancia: distancia}}

      _ ->
        {:error, "distancia no numerica"}
    end
  end
end
```

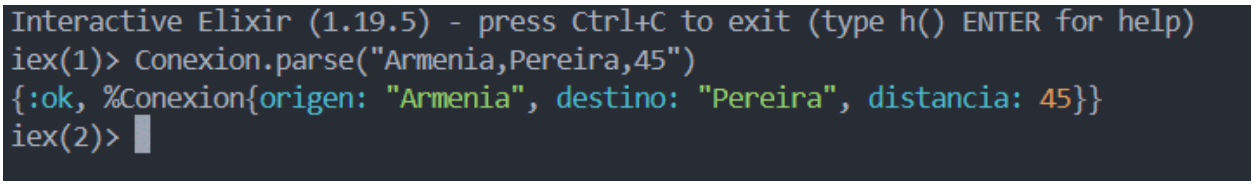
Figura 5. *Código fuente de conexion.exs.*

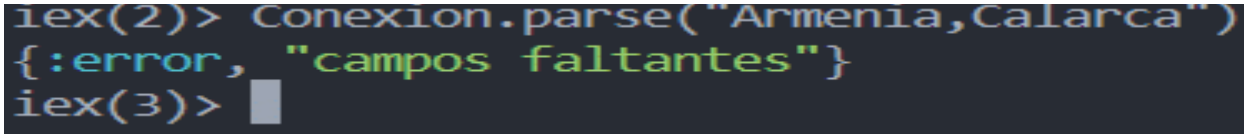
Prueba realizada:

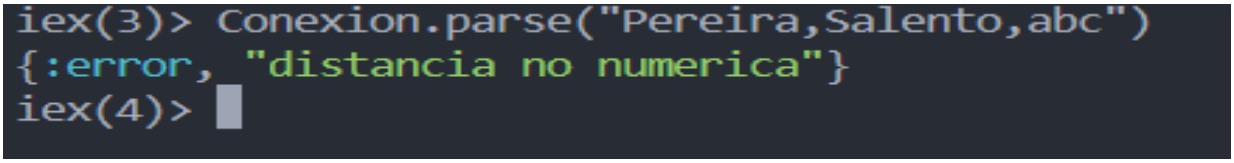
```
iex> Conexion.parse("Armenia,Pereira,45")
```

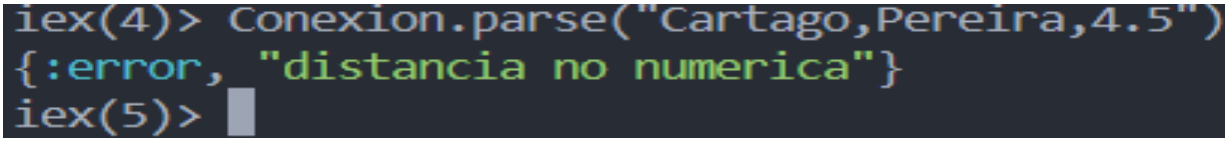
```
{:ok, %Conexion{origen: "Armenia", destino: "Pereira", distancia: 45}}
iex> Conexion.parse("Armenia,Calarca")
{:error, "campos faltantes"}
iex> Conexion.parse("Pereira,Salento,abc")
{:error, "distancia no numerica"}
iex> Conexion.parse("Cartago,Pereira,4.5")
{:error, "distancia no numerica"}
iex> Conexion.parse("Cartago,Tulua,40,extra")
{:error, "campos faltantes"}
```

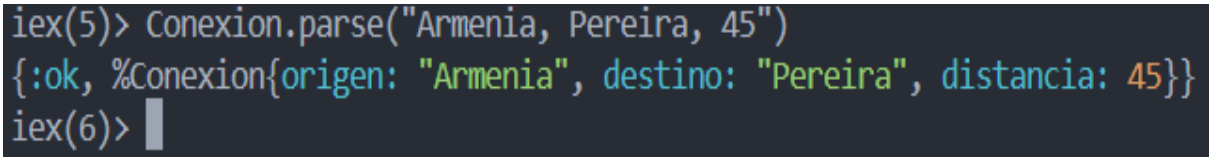
Figura 6. *Pruebas de Conexion.parse/1 en iex (salida real).*

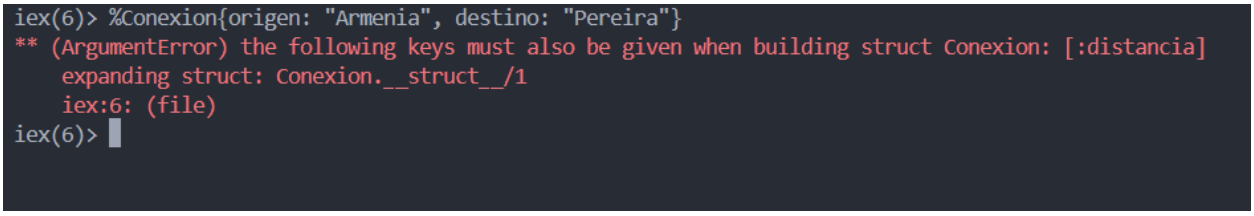












2.4.2 Lector (lector.exs)

Lee los archivos de entrada de forma perezosa. La función general `leer/2` recibe la ruta y la función de conversión de cada línea, por lo que sirve para ambos archivos: `leer_conexiones/1` usa `Conexion.parse/1` y `leer_consultas/1` usa `parse_consulta/1`.

El pipeline numera las líneas con `Stream.with_index(1)` **antes** de filtrar, porque el parcial indica que los números de línea cuentan todas las líneas, incluidos comentarios y vacías. Luego quita el salto de línea, descarta vacías y comentarios con `Stream.reject` y convierte cada línea. Nada se ejecuta hasta `Enum.reduce`, que consume el stream y separa válidas y errores con `clasificar/2`.

Si el archivo no existe, devuelve `{:error, "El archivo ... no existe"}` en lugar de dejar que `File.stream!` lance una excepción.

```
defmodule Lector do
  @moduledoc """
  Lee los archivos de entrada de forma perezosa, línea por línea.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def leer_conexiones(ruta), do: leer(ruta, &Conexion.parse/1)

  def leer_consultas(ruta), do: leer(ruta, &parse_consulta/1)

  defp leer(ruta, convertir) do
    if File.exists?(ruta) do
      {validas, errores} =
        ruta
        |> File.stream!()
        |> Stream.with_index(1)
        |> Stream.map(fn {linea, numero} -> {numero, String.trim(linea)} end)
        |> Stream.reject(fn {_numero, linea} -> ignorar?(linea) end)
        |> Stream.map(fn {numero, linea} -> {numero, linea, convertir.(linea)} end)
        |> Enum.reduce({[], []}, &clasificar/2)

      {:ok, Enum.reverse(validas), Enum.reverse(errores)}
    else
      {:error, "El archivo #{ruta} no existe"}
    end
  end

  defp parse_consulta(linea) do
    case linea |> String.split(",") |> Enum.map(&String.trim/1) do
      [origen, destino] when origen != "" and destino != "" -> {:ok, {origen, destino}}
      _ -> {:error, "consulta mal formada"}
    end
  end

  defp ignorar?(""), do: true
  defp ignorar?(linea), do: String.starts_with?(linea, "#")

  defp clasificar({numero, linea, {:ok, dato}}, {validas, errores}) do
    {[{numero, linea, dato} | validas], errores}
  end

  defp clasificar({numero, linea, {:error, motivo}}, {validas, errores}) do
    {validas, [{numero, linea, motivo} | errores]}
  end
end
```

Figura 7. *Código fuente de lector.exs.*

Prueba realizada:

```
iex> Lector.leer_conexiones("entradas/conexiones.csv")
```

```
{:ok,
 [
   {2, "Armenia,Pereira,45", %Conexion{origen: "Armenia", destino: "Pereira", distancia:
45}},
   {3, "Pereira,Manizales,52", %Conexion{origen: "Pereira", destino: "Manizales",
distancia: 52}},
   {4, "Armenia,Cartago,60", %Conexion{origen: "Armenia", destino: "Cartago", distancia:
60}},
   {5, "Cartago,Pereira,25", %Conexion{origen: "Cartago", destino: "Pereira", distancia:
25}},
   {6, "Pereira,Armenia,45", %Conexion{origen: "Pereira", destino: "Armenia", distancia:
45}},
   {7, "Manizales,Manizales,0", %Conexion{origen: "Manizales", destino: "Manizales",
distancia: 0}},
   {8, "Cartago,Pereira,30", %Conexion{origen: "Cartago", destino: "Pereira", distancia:
30}}
 ], []}
```

Figura 8. *Lectura del archivo de ejemplo del parcial (salida real, antes de agregar las líneas de prueba).*



2.4.3 Depuracion (depuracion.exs)

Implementa la validación previa con recursión propia, sin `Enum` ni `Stream`. Recorre la lista de conexiones como una fila de inspección: toma la primera, decide si se conserva o se descarta y se llama a sí misma con el resto.

Una conexión de una ciudad consigo misma se detecta con pattern matching, usando la misma variable en ambos campos: `%Conexion{origen: ciudad, destino: ciudad}`. Para las demás, `buscar_par/2` recorre recursivamente las conexiones ya conservadas y `mismo_par?/2` compara en ambos sentidos. Si el par ya existe, `motivo_duplicado/3` decide entre "repetida" y "distancia distinta", conservando siempre la primera aparición.

Como no se puede usar `Enum.reverse`, `invertir/2` es una versión propia con recursión de cola.

```
defmodule Depuracion do
  @moduledoc """
  Depura la lista de conexiones antes de construir el grafo, usando solo
  recursión propia (sin Enum ni Stream):
  - descarta las conexiones de una ciudad consigo misma;
  - descarta las repetidas (mismo sentido o sentido contrario);
  - si un par aparece con otra distancia, conserva la primera.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def depurar(conexiones), do: depurar(conexiones, [], [])

  defp depurar([], conservadas, descartes) do
    {invertir(conservadas, []), invertir(descartes, [])}
  end

  defp depurar(
         [{numero, linea, %Conexion{origen: ciudad, destino: ciudad}} | resto],
         conservadas,
         descartes
       ) do
    motivo = "conecta una ciudad consigo misma; se descarta"
    depurar(resto, conservadas, [{numero, linea, motivo} | descartes])
  end

  defp depurar([{numero, linea, conexion} = actual | resto], conservadas, descartes) do
    case buscar_par(conservadas, conexion) do
      nil ->
        depurar(resto, [actual | conservadas], descartes)

      {numero_previo, _linea_previa, previa} ->
        motivo = motivo_duplicado(conexion, previa, numero_previo)
        depurar(resto, conservadas, [{numero, linea, motivo} | descartes])
    end
  end

  defp buscar_par([], _conexion), do: nil

  defp buscar_par([{_numero, _linea, previa} = item | resto], conexion) do
    if mismo_par?(previa, conexion) do
      item
    else
      buscar_par(resto, conexion)
    end
  end

  defp mismo_par?(a, b) do
    (a.origen == b.origen and a.destino == b.destino) or
      (a.origen == b.destino and a.destino == b.origen)
  end

  defp motivo_duplicado(conexion, previa, numero_previo) do
    if conexion.distancia == previa.distancia do
      "repetida (línea #{numero_previo}); se descarta"
    else
      "distancia distinta a la de la línea #{numero_previo} (#{previa.distancia}); " <>
        "se conserva la de la línea #{numero_previo}"
    end
  end

  defp invertir([], acumulado), do: acumulado
```

```
  defp invertir([cabeza | cola], acumulado), do: invertir(cola, [cabeza | acumulado])
end
```

Figura 9. *Código fuente de depuracion.exs.*

2.4.4 Grafo (grafo.exs)

Convierte la lista de conexiones depuradas en un mapa `ciudad => [{vecino, distancia}]`, que permite responder de inmediato "¿a qué ciudades puedo ir desde aquí?".

Usa el patrón visto en clase `Enum.reduce` + `Map.update/4`. Por cada conexión agrega dos entradas, una en cada sentido, porque las conexiones se recorren en ambos sentidos. Los vecinos se agregan al final (`++`) para conservar el orden del archivo.

```
defmodule Grafo do
  @moduledoc """
  Construye el grafo de ciudades a partir de las conexiones depuradas.
  El grafo es un mapa: ciudad => [{vecino, distancia}].
  Las conexiones se pueden recorrer en ambos sentidos.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def construir(conexiones) do
    Enum.reduce(conexiones, %{}, fn {_numero, _linea, conexion}, grafo ->
      grafo
      |> agregar_vecino(conexion.origen, conexion.destino, conexion.distancia)
      |> agregar_vecino(conexion.destino, conexion.origen, conexion.distancia)
    end)
  end

  def vecinos(grafo, ciudad), do: Map.get(grafo, ciudad, [])

  def existe?(grafo, ciudad), do: Map.has_key?(grafo, ciudad)

  defp agregar_vecino(grafo, ciudad, vecino, distancia) do
    Map.update(grafo, ciudad, [{vecino, distancia}], fn vecinos ->
      vecinos ++ [{vecino, distancia}]
    end)
  end
end
```

Figura 10. *Código fuente de grafo.exs.*

Prueba realizada:

```
iex> grafo = Grafo.construir(conservadas)
%{
  "Armenia" => [{"Pereira", 45}, {"Cartago", 60}],
  "Cartago" => [{"Armenia", 60}, {"Pereira", 25}],
  "Manizales" => [{"Pereira", 52}],
  "Pereira" => [{"Armenia", 45}, {"Manizales", 52}, {"Cartago", 25}]
}
iex> Grafo.vecinos(grafo, "Pereira")
[{"Armenia", 45}, {"Manizales", 52}, {"Cartago", 25}]
iex> Grafo.vecinos(grafo, "Bogota")
[]
```

Figura 11. *Grafo construido a partir de las conexiones depuradas (salida real).*



2.4.5 Alcance (alcance.exs)

Construye el Stream infinito de conjuntos de ciudades alcanzables. Su funcionamiento se explica en detalle en la sección 2.6.

```
defmodule Alcance do
  @moduledoc """
  Calcula, con un Stream infinito, los conjuntos de ciudades alcanzables
  desde un origen. El paso 0 contiene solo el origen y cada paso agrega los
  vecinos de las ciudades del paso anterior. Se detiene en el primer paso repetido.
```

```
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def secuencia(grafo, origen) do
    Stream.iterate([origen], fn conjunto -> siguiente_paso(grafo, conjunto) end)
  end

  def pasos(grafo, origen) do
    pares_distintos =
      secuencia(grafo, origen)
      |> Stream.chunk_every(2, 1)
      |> Stream.take_while(fn [anterior, actual] -> anterior != actual end)
      |> Enum.to_list()

    [[origen] | Enum.map(pares_distintos, fn [_anterior, actual] -> actual end)]
  end

  def alcanzable?(pasos, destino), do: destino in List.last(pasos)

  defp siguiente_paso(grafo, conjunto) do
    vecinos =
      Enum.flat_map(conjunto, fn ciudad ->
        Enum.map(Grafo.vecinos(grafo, ciudad), fn {vecino, _distancia} -> vecino end)
      end)

    (conjunto ++ vecinos)
    |> Enum.uniq()
    |> Enum.sort()
  end
end
```

Figura 12. *Código fuente de alcance.exs.*

2.4.6 Ruta (ruta.exs)

Define el struct `%Ruta{ciudades, distancia}` exigido por el parcial y las funciones que la describen: `numero_ciudades/1` (con `length`), `recorrido/1` (con `Enum.join(ciudades, " -> ")`) y `a_texto/1`, que reutiliza las dos anteriores. De este struct se generan tanto la línea de pantalla como la del archivo.

```
defmodule Ruta do
  @moduledoc """
  Representa una ruta encontrada: la lista de ciudades en orden
  y la distancia total en kilómetros.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  @enforce_keys [:ciudades, :distancia]
  defstruct [:ciudades, :distancia]

  def numero_ciudades(%Ruta{ciudades: ciudades}), do: length(ciudades)

  def recorrido(%Ruta{ciudades: ciudades}), do: Enum.join(ciudades, " -> ")

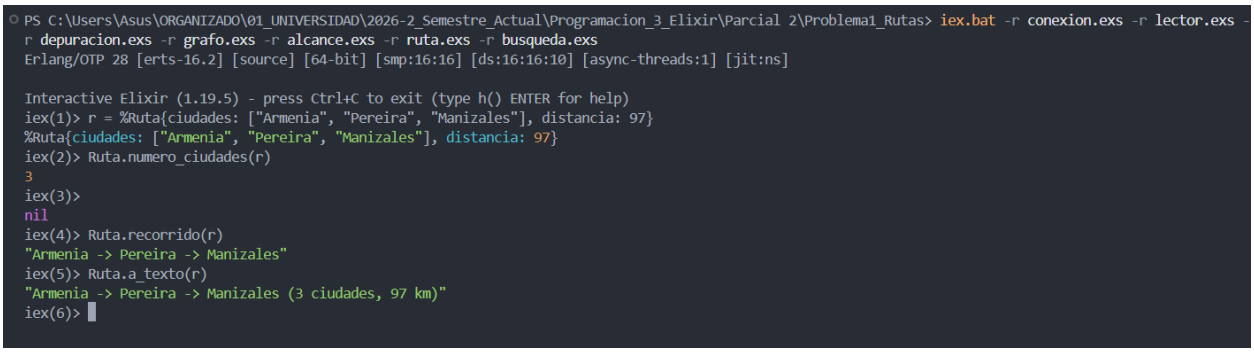
  def a_texto(%Ruta{} = ruta) do
    "#{recorrido(ruta)} (#{numero_ciudades(ruta)} ciudades, #{ruta.distancia} km)"
  end
end
```

Figura 13. *Código fuente de ruta.exs.*

Prueba realizada:

```
iex> r = %Ruta{ciudades: ["Armenia", "Pereira", "Manizales"], distancia: 97}
iex> Ruta.numero_ciudades(r)
3
iex> Ruta.recorrido(r)
"Armenia -> Pereira -> Manizales"
```

Figura 14. *Pruebas de las funciones del struct Ruta (salida real).*



2.4.7 Busqueda (busqueda.exs)

Encuentra todas las rutas con recursión y backtracking. Su funcionamiento se explica en detalle en la sección 2.7.

```
defmodule Busqueda do
  @moduledoc """
  Encuentra todas las rutas entre dos ciudades con recursión y backtracking.
  Una ciudad no puede aparecer dos veces en la misma ruta.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def rutas(grafo, origen, destino), do: buscar(grafo, origen, destino, [origen], 0)

  defp buscar(_grafo, destino, destino, camino, distancia) do
    [%Ruta{ciudades: camino, distancia: distancia}]
  end

  defp buscar(grafo, actual, destino, camino, distancia) do
    explorar(Grafo.vecinos(grafo, actual), grafo, destino, camino, distancia)
  end

  defp explorar([], _grafo, _destino, _camino, _distancia), do: []

  defp explorar([{vecino, km} | resto], grafo, destino, camino, distancia) do
    rutas_por_vecino =
      if vecino in camino do
        []
      else
        buscar(grafo, vecino, destino, camino ++ [vecino], distancia + km)
      end

    rutas_por_vecino ++ explorar(resto, grafo, destino, camino, distancia)
  end
```

```
end
```

Figura 15. *Código fuente de busqueda.exs.*

2.4.8 Reporte (reporte.exs)

Transforma los resultados en líneas de texto sin imprimir ni escribir nada (funciones puras). `lineas_errores/1` sirve tanto para los errores de formato como para los descartes de la depuración, porque ambos tienen la forma `{numero, linea, motivo}`.

`mas_cortas/1` devuelve **todas** las rutas con el menor número de ciudades (en caso de empate aparecen todas) y `menor_distancia/1` usa `Enum.min_by`. La función privada `detalle/1` tiene una cláusula por cada estado de la consulta: ciudad inexistente, sin rutas o con rutas.

```
defmodule Reporte do
  @moduledoc """
  Construye las líneas de texto de cada archivo de salida. No escribe ni
  imprime nada: solo transforma datos en texto.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def lineas_errores(errores) do
    Enum.map(errores, fn {numero, linea, motivo} -> "Línea #{numero}: #{linea} ->
#{motivo}" end)
  end

  def lineas_alcance(%{origen: origen, pasos: nil}) do
    ["=== Alcance desde #{origen} ===", "No se calcula: la consulta tiene ciudades que no
existen", ""]
  end

  def lineas_alcance(%{origen: origen, pasos: pasos}) do
    detalle =
      pasos
      |> Enum.with_index()
      |> Enum.map(fn {paso, numero} -> "Paso #{numero}: #{Enum.join(paso, ", ")}" end)

    ["=== Alcance desde #{origen} ==="] ++
      detalle ++ ["Se estabiliza en el paso #{length(pasos) - 1}", ""]
  end

  def lineas_consulta(%{origen: origen, destino: destino} = resultado) do
    ["=== #{origen} -> #{destino} ==="] ++ detalle(resultado) ++ [""]
  end

  def mas_cortas(rutas) do
    minimo = rutas |> Enum.map(&Ruta.numero_ciudades/1) |> Enum.min()
    Enum.filter(rutas, fn ruta -> Ruta.numero_ciudades(ruta) == minimo end)
  end

  def menor_distancia(rutas), do: Enum.min_by(rutas, fn ruta -> ruta.distancia end)

  defp detalle(%{estado: :ciudad_inexistente, faltantes: faltantes}) do
    ["No existe en el grafo: #{Enum.join(faltantes, ", ")}"]
  end

  defp detalle(%{estado: :sin_rutas}) do
    ["Sin rutas: el destino no es alcanzable desde el origen", "Número de rutas: 0"]
  end
```

```
  defp detalle(%{estado: :con_rutas, rutas: rutas}) do
    numeradas =
      rutas
      |> Enum.with_index(1)
      |> Enum.map(fn {ruta, numero} -> "Ruta #{numero}: #{Ruta.a_texto(ruta)}" end)

    cortas = rutas |> mas_cortas() |> Enum.map(&Ruta.recorrido/1) |> Enum.join(" | ")
    menor = menor_distancia(rutas)

    numeradas ++
      [
        "Número de rutas: #{length(rutas)}",
        "Ruta más corta: #{cortas}",
        "Ruta de menor distancia: #{Ruta.recorrido(menor)} (#{menor.distancia} km)"
      ]
  end
end
```

Figura 16. *Código fuente de reporte.exs.*

2.4.9 Salida (salida.exs)

`escribir_lineas/2` agrega el salto de línea a cada línea y la escribe apenas se genera con `Enum.into(File.stream!(ruta, [:write]))`, la misma técnica del ejemplo del anonimizador visto en clase. El modo `[:write]` crea el archivo o lo reemplaza si ya existe, y nunca se construye un único texto con toda la salida.

```
defmodule Salida do
  @moduledoc """
  Escribe resultados en archivos línea por línea y los muestra en pantalla.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def escribir_lineas(lineas, ruta) do
    lineas
    |> Stream.map(fn linea -> linea <> "\n" end)
    |> Enum.into(File.stream!(ruta, [:write]))

    :ok
  end

  def mostrar_lineas(lineas), do: Enum.each(lineas, &IO.puts/1)
end
```

Figura 17. *Código fuente de salida.exs.*

2.4.10 Programa (programa.exs)

Contiene `main/0`, que coordina todo el flujo. Con `with` encadena la lectura de ambos archivos; si alguno no existe, muestra el mensaje de error y termina sin excepción.

`resolver/3` decide qué hacer con cada consulta: si una ciudad no existe en el grafo lo informa; si existe, calcula el alcance y, solo si el destino es alcanzable, ejecuta la búsqueda recursiva. `publicar/2` escribe cada archivo de salida y muestra las mismas líneas en pantalla.

```
defmodule Programa do
  @moduledoc """
  Problema 1: rutas entre ciudades. Orquesta la lectura, la depuración,
  el grafo, el alcance, la búsqueda y la escritura de resultados.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  @entradas Path.join(__DIR__, "entradas")
  @salidas Path.join(__DIR__, "salidas")

  def main do
    with {:ok, validas, errores} <- Lector.leer_conexiones(Path.join(@entradas,
"conexiones.csv")),
         {:ok, consultas, errores_consultas} <-
Lector.leer_consultas(Path.join(@entradas, "consultas.txt")) do
      File.mkdir_p!(@salidas)

      {conservadas, descartes} = Depuracion.depurar(validas)
      grafo = Grafo.construir(conservadas)

      resultados =
        Enum.map(consultas, fn {_numero, _linea, {origen, destino}} ->
          resolver(grafo, origen, destino)
        end)

      publicar(Reporte.lineas_errores(errores), "errores_conexiones.txt")
      publicar(Reporte.lineas_errores(descartes), "depuracion_conexiones.txt")
      publicar(Stream.flat_map(resultados, &Reporte.lineas_alcance/1), "alcance.txt")
      publicar(Stream.flat_map(resultados, &Reporte.lineas_consulta/1),
"rutas_encontradas.txt")
      mostrar_errores_consultas(errores_consultas)
    else
      {:error, mensaje} -> IO.puts(:standard_error, mensaje)
    end
  end

  defp resolver(grafo, origen, destino) do
    faltantes = [origen, destino] |> Enum.uniq() |> Enum.reject(&Grafo.existe?(grafo,
&1))

    if faltantes != [] do
      %{origen: origen, destino: destino, estado: :ciudad_inexistente, faltantes:
faltantes, pasos: nil}
    else
      pasos = Alcance.pasos(grafo, origen)

      if Alcance.alcanzable?(pasos, destino) do
        rutas = Busqueda.rutas(grafo, origen, destino)
        %{origen: origen, destino: destino, estado: :con_rutas, pasos: pasos, rutas:
rutas}
      else
        %{origen: origen, destino: destino, estado: :sin_rutas, pasos: pasos}
      end
    end
  end

  defp publicar(lineas, nombre) do
    Salida.escribir_lineas(lineas, Path.join(@salidas, nombre))
    IO.puts("\n----- #{nombre} -----")

    if Enum.empty?(lineas) do
      IO.puts("(sin registros)")
    else
      Salida.mostrar_lineas(lineas)
```

```
    end
  end

  defp mostrar_errores_consultas([]), do: :ok

  defp mostrar_errores_consultas(errores) do
    IO.puts("\n----- consultas ignoradas -----")
    errores |> Reporte.lineas_errores() |> Salida.mostrar_lineas()
  end
end

Programa.main()
```

Figura 18. *Código fuente de programa.exs.*

2.5 Validación con recursión: depuración de conexiones

2.5.1 Casos base y casos recursivos

Función	Caso base	Caso recursivo	¿Qué cambia en cada llamada?
`depurar/3`	Lista vacía: invierte y devuelve `{conservadas, descartes}`	Procesa la primera conexión y se llama con `resto`	La fila pierde un elemento; crece conservadas o descartes
`buscar_par/2`	Lista vacía: `nil` (no se encontró)	Si la primera no es el par, se llama con `resto`	La lista de conservadas se recorre un elemento por vez
`invertir/2`	Lista vacía: devuelve el acumulado	Pasa la cabeza al acumulado (recursión de cola)	La lista se acorta y el acumulado crece

Tabla 4. *Estructura recursiva de la depuración.*

2.5.2 Prueba de escritorio

Llamada	Conexión actual	¿Misma ciudad?	buscar_par en conservadas	Decisión
1	L2 Armenia-Pereira 45	No	[] → nil	Conservar
2	L3 Pereira-Manizales 52	No	[L2] → nil	Conservar
3	L4 Armenia-Cartago 60	No	[L3, L2] → nil	Conservar
4	L5 Cartago-Pereira 25	No	[L4, L3, L2] → nil	Conservar
5	L6 Pereira-Armenia 45	No	Encuentra L2 (sentido contrario); 45 = 45	Descartar: repetida (línea 2)
6	L7 Manizales-Manizales 0	**Sí**	(no se busca)	Descartar: ciudad consigo misma
7	L8 Cartago-Pereira 30	No	Encuentra L5; 30 ≠ 25	Descartar: distancia distinta; se conserva la línea 5
8	(lista vacía)	—	—	Caso base: `{[L2,L3,L4,L5], [D6,D7,D8]}`

Tabla 5. *Prueba de escritorio de Depuracion.depurar/1 con el archivo del parcial.*

2.5.3 Árbol de llamadas recursivas

```
depurar([L2,L3,L4,L5,L6,L7,L8], [], [])
│ L2: buscar_par([]) → nil → CONSERVAR
└─ depurar([L3..L8], [L2], [])
   │ L3: buscar_par([L2]) → no coincide → buscar_par([]) → nil → CONSERVAR
   └─ depurar([L4..L8], [L3,L2], [])
      │ L4: recorre L3, L2 → nil → CONSERVAR
      └─ depurar([L5..L8], [L4,L3,L2], [])
         │ L5: recorre L4, L3, L2 → nil → CONSERVAR
         └─ depurar([L6,L7,L8], [L5,L4,L3,L2], [])
            │ L6 Pereira-Armenia: recorre L5, L4, L3 → L2 coincide al revés; 45 == 45
            │    → "repetida (línea 2); se descarta"
            └─ depurar([L7,L8], [L5,L4,L3,L2], [D6])
               │ L7: encaja con %Conexion{origen: ciudad, destino: ciudad}
               │    → "conecta una ciudad consigo misma; se descarta"
               └─ depurar([L8], [L5,L4,L3,L2], [D7,D6])
                  │ L8 Cartago-Pereira: L5 coincide; 30 ≠ 25
                  │    → "distancia distinta a la de la línea 5 (25); se conserva la de
la línea 5"
                  └─ depurar([], [L5,L4,L3,L2], [D8,D7,D6])
                     ← CASO BASE: invertir ambas listas
                       → {[L2,L3,L4,L5], [D6,D7,D8]}

invertir([1, 2, 3], [])
└─ invertir([2, 3], [1])
   └─ invertir([3], [2, 1])
      └─ invertir([], [3, 2, 1])  ← CASO BASE → [3, 2, 1]
```

Figura 19. *Árbol de llamadas de depurar/3 e invertir/2.*

2.5.4 Resultado obtenido

```
----- depuracion_conexiones.txt -----
Línea 6: Pereira,Armenia,45 -> repetida (línea 2); se descarta
Línea 7: Manizales,Manizales,0 -> conecta una ciudad consigo misma; se descarta
Línea 8: Cartago,Pereira,30 -> distancia distinta a la de la línea 5 (25); se conserva la
de la línea 5
```

Figura 20. *Contenido real de depuracion_conexiones.txt. Coincide con el ejemplo del parcial.*

2.6 Streams infinitos: ciudades alcanzables

2.6.1 Funcionamiento

`Alcance.secuencia/2` construye con `Stream.iterate/2` una secuencia infinita de conjuntos: el paso 0 es `[origen]` y cada paso siguiente se obtiene aplicando `siguiente_paso/2` al anterior. Esa función une las ciudades actuales con los vecinos de cada una (`Enum.flat_map`), elimina repetidos (`Enum.uniq`) y ordena alfabéticamente (`Enum.sort`). Ordenar garantiza que dos pasos con las mismas ciudades sean idénticos y puedan compararse con `!=`.

Para detectar la estabilización sin calcular más de lo necesario, `pasos/2` agrupa los pasos de dos en dos con `Stream.chunk_every(2, 1)` y toma pares con `Stream.take_while` mientras sean distintos. El stream se consume solo en `Enum.to_list`, y el último paso calculado es el primero que se repite, como pide el enunciado.

Si el destino no está en el último paso (`alcanzable?/2`), la consulta se reporta como "sin rutas" y **no se ejecuta la búsqueda recursiva**.

2.6.2 Prueba de escritorio (origen: Armenia)

Paso	Conjunto anterior	Vecinos encontrados	Resultado (uniq + sort)
0	(valor inicial)	—	[Armenia]
1	[Armenia]	Pereira, Cartago	[Armenia, Cartago, Pereira]
2	[Armenia, Cartago, Pereira]	Pereira, Cartago, Armenia, Pereira, Armenia, Manizales, Cartago	[Armenia, Cartago, Manizales, Pereira]
3	[Armenia, Cartago, Manizales, Pereira]	(ninguno nuevo)	[Armenia, Cartago, Manizales, Pereira] → igual al paso 2

Tabla 6. *Generación de los pasos del Stream infinito.*

```
Pares que revisa take_while:
[p0, p1]  ¿distintos? sí → se toma
[p1, p2]  ¿distintos? sí → se toma
[p2, p3]  ¿distintos? NO → se detiene (p3 es el último paso calculado)

pasos = [p0, p1, p2]  →  "Se estabiliza en el paso 2"
```

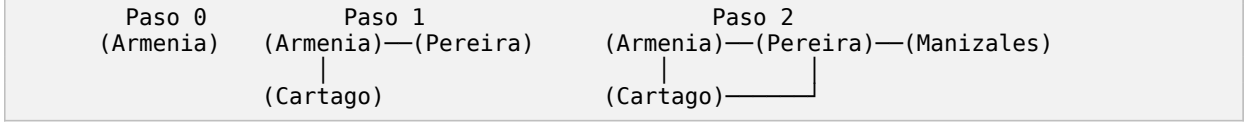


Figura 21. *Detección de la estabilización y expansión del alcance desde Armenia.*

2.6.3 Resultado obtenido

```
=== Alcance desde Armenia ===
Paso 0: Armenia
Paso 1: Armenia, Cartago, Pereira
Paso 2: Armenia, Cartago, Manizales, Pereira
Se estabiliza en el paso 2
```

Figura 22. *Fragmento real de alcance.txt. Coincide con el ejemplo del parcial.*

```
iex(4)> clear()
iex(5)> Alcance.secuencia(grafo, "Armenia") |> Enum.take(6)
[
  ["Armenia"],
  ["Armenia", "Cartago", "Pereira"],
  ["Armenia", "Cartago", "Manizales", "Pereira"],
  ["Armenia", "Cartago", "Manizales", "Pereira"],
  ["Armenia", "Cartago", "Manizales", "Pereira"],
  ["Armenia", "Cartago", "Manizales", "Pereira"]
]
iex(6)>
iex(6)> pasos = Alcance.pasos(grafo, "Armenia")
[
  ["Armenia"],
  ["Armenia", "Cartago", "Pereira"],
  ["Armenia", "Cartago", "Manizales", "Pereira"]
]
iex(7)> Alcance.alcanzable?(pasos, "Manizales")
true
iex(8)> Alcance.alcanzable?(pasos, "Bogota")
false
iex(9)>
```

2.7 Búsqueda principal: todas las rutas

2.7.1 Funcionamiento

La búsqueda usa dos funciones que se llaman entre sí. `buscar/5` representa estar parado en una ciudad: si la ciudad actual es el destino (caso base, detectado repitiendo la variable `destino` en el molde), devuelve una lista con un `%Ruta{}`; si no, pasa los vecinos a `explorar/5`. Esta última recorre la lista de vecinos con recursión propia: si el vecino ya está en el camino lo omite (así ninguna ciudad se repite y se evitan ciclos); si no, entra en él con una llamada a `buscar` con el camino ampliado y la distancia acumulada, y después continúa con el resto de vecinos.

El backtracking ocurre de forma natural: al continuar con `explorar(resto, ...)` se usan el camino y la distancia originales, porque los datos en Elixir son inmutables y `camino ++ [vecino]` creó una lista nueva sin modificar la anterior.

Función	Estado que recibe	Caso base	Caso recursivo
`buscar/5`	ciudad actual, destino, camino, distancia	actual == destino → `[%Ruta{}]`	Llama a `explorar` con los vecinos
`explorar/5`	lista de vecinos pendientes	`[]` → `[]`	Entra al vecino (llama a `buscar`) y se llama con `resto`

Tabla 7. *Estructura recursiva de la búsqueda de rutas.*

2.7.2 Árbol de llamadas: Armenia → Manizales

```
buscar(Armenia, camino=[A], 0)                  ¿llegué? no → explorar [Pereira 45,
Cartago 60]
├─ Pereira: ¿en [A]? no → ENTRAR
│  buscar(Pereira, [A,P], 45)                   ¿llegué? no → explorar [Armenia,
Manizales, Cartago]
│  ├─ Armenia: en el camino → []
│  ├─ Manizales: no → ENTRAR
│  │  buscar(Manizales, [A,P,M], 97)            ¿llegué? SÍ → [Ruta A-P-M, 97 km]
│  └─ Cartago: no → ENTRAR
│     buscar(Cartago, [A,P,C], 70)             ¿llegué? no → explorar [Armenia, Pereira]
│     ├─ Armenia: en el camino → []
```

```
│     └─ Pereira: en el camino → []              (callejón sin salida: vuelve atrás)
└─ Cartago: ¿en [A]? no → ENTRAR
   buscar(Cartago, [A,C], 60)                   ¿llegué? no → explorar [Armenia, Pereira]
   ├─ Armenia: en el camino → []
   └─ Pereira: no → ENTRAR
      buscar(Pereira, [A,C,P], 85)              ¿llegué? no → explorar [Armenia,
Manizales, Cartago]
      ├─ Armenia: en el camino → []
      ├─ Manizales: no → ENTRAR
      │  buscar(Manizales, [A,C,P,M], 137)      ¿llegué? SÍ → [Ruta A-C-P-M, 137 km]
      └─ Cartago: en el camino → []

Resultado: [Ruta A-P-M (97 km), Ruta A-C-P-M (137 km)]
```

Figura 23. *Árbol de llamadas recursivas de la búsqueda Armenia → Manizales.*

2.7.3 Prueba de escritorio

Llamada	Ciudad actual	Camino	Distancia	Decisión
1	Armenia	[A]	0	No es destino; explora Pereira y Cartago
2	Pereira	[A, P]	45	No es destino; Armenia ya visitada
3	Manizales	[A, P, M]	97	**Destino**: ruta 1
4	Cartago	[A, P, C]	70	Todos los vecinos visitados: sin rutas
5	Cartago	[A, C]	60	Armenia visitada; entra a Pereira
6	Pereira	[A, C, P]	85	Armenia visitada; entra a Manizales; Cartago visitada
7	Manizales	[A, C, P, M]	137	**Destino**: ruta 2

Tabla 8. *Prueba de escritorio de Busqueda.rutas/3.*

2.8 Resultados obtenidos

Salida real del programa completo con los datos de la sección 2.3, ejecutado con el comando de la sección 1.4:

```
----- errores_conexiones.txt -----
Línea 11: Armenia,Calarca -> campos faltantes
Línea 12: Pereira,Salento,abc -> distancia no numerica

----- depuracion_conexiones.txt -----
Línea 6: Pereira,Armenia,45 -> repetida (línea 2); se descarta
Línea 7: Manizales,Manizales,0 -> conecta una ciudad consigo misma; se descarta
Línea 8: Cartago,Pereira,30 -> distancia distinta a la de la línea 5 (25); se conserva la
de la línea 5

----- alcance.txt -----
=== Alcance desde Armenia ===
Paso 0: Armenia
Paso 1: Armenia, Cartago, Pereira
Paso 2: Armenia, Cartago, Manizales, Pereira
Se estabiliza en el paso 2

=== Alcance desde Cartago ===
Paso 0: Cartago
Paso 1: Armenia, Cartago, Pereira
```

```
Paso 2: Armenia, Cartago, Manizales, Pereira
Se estabiliza en el paso 2

=== Alcance desde Armenia ===
Paso 0: Armenia
Paso 1: Armenia, Cartago, Pereira
Paso 2: Armenia, Cartago, Manizales, Pereira
Se estabiliza en el paso 2

=== Alcance desde Armenia ===
No se calcula: la consulta tiene ciudades que no existen

----- rutas_encontradas.txt -----
=== Armenia -> Manizales ===
Ruta 1: Armenia -> Pereira -> Manizales (3 ciudades, 97 km)
Ruta 2: Armenia -> Cartago -> Pereira -> Manizales (4 ciudades, 137 km)
Número de rutas: 2
Ruta más corta: Armenia -> Pereira -> Manizales
Ruta de menor distancia: Armenia -> Pereira -> Manizales (97 km)

=== Cartago -> Manizales ===
Ruta 1: Cartago -> Armenia -> Pereira -> Manizales (4 ciudades, 157 km)
Ruta 2: Cartago -> Pereira -> Manizales (3 ciudades, 77 km)
Número de rutas: 2
Ruta más corta: Cartago -> Pereira -> Manizales
Ruta de menor distancia: Cartago -> Pereira -> Manizales (77 km)

=== Armenia -> Bogota ===
Sin rutas: el destino no es alcanzable desde el origen
Número de rutas: 0

=== Armenia -> Cali ===
No existe en el grafo: Cali

----- consultas ignoradas -----
Línea 6: Pereira -> consulta mal formada
```

Figura 24. *Salida real completa del problema 1 en pantalla.*

```
PS C:\Users\Asus\ORGANIZADO\01_UNIVERSIDAD\2026-2_Semestre_Actual\Programacion_3_Elixir\Parcial 2\Problema1_Rutas> elixir -r conexion.exs -r lector.exs -r
 depuracion.exs -r grafo.exs -r alcance.exs -r ruta.exs -r busqueda.exs -r reporte.exs -r salida.exs programa.exs

----- errores_conexiones.txt -----
Línea 11: Armenia,Calarca -> campos faltantes
Línea 12: Pereira,Salento,abc -> distancia no numerica

----- depuracion_conexiones.txt -----
Línea 6: Pereira,Armenia,45 -> repetida (línea 2); se descarta
Línea 7: Manizales,Manizales,0 -> conecta una ciudad consigo misma; se descarta
Línea 8: Cartago,Pereira,30 -> distancia distinta a la de la línea 5 (25); se conserva la de la línea 5

----- alcance.txt -----
=== Alcance desde Armenia ===
Paso 0: Armenia
Paso 1: Armenia, Cartago, Pereira
Paso 2: Armenia, Cartago, Manizales, Pereira
Se estabiliza en el paso 2

=== Alcance desde Cartago ===
Paso 0: Cartago
Paso 1: Armenia, Cartago, Pereira
Paso 2: Armenia, Cartago, Manizales, Pereira
Se estabiliza en el paso 2

=== Alcance desde Armenia ===
Paso 0: Armenia
Paso 1: Armenia, Cartago, Pereira
Paso 2: Armenia, Cartago, Manizales, Pereira
Se estabiliza en el paso 2

=== Alcance desde Armenia ===
No se calcula: la consulta tiene ciudades que no existen

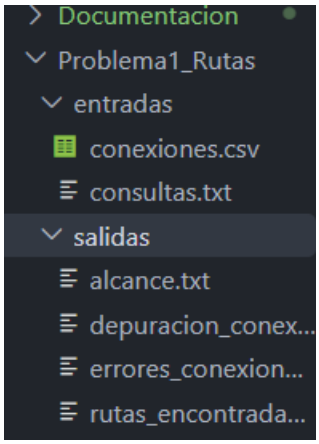

----- rutas_encontradas.txt -----
=== Armenia -> Manizales ===
Ruta 1: Armenia -> Pereira -> Manizales (3 ciudades, 97 km)
Ruta 2: Armenia -> Cartago -> Pereira -> Manizales (4 ciudades, 137 km)
Número de rutas: 2
Ruta más corta: Armenia -> Pereira -> Manizales
Ruta de menor distancia: Armenia -> Pereira -> Manizales (97 km)

=== Cartago -> Manizales ===
Ruta 1: Cartago -> Armenia -> Pereira -> Manizales (4 ciudades, 157 km)
Ruta 2: Cartago -> Pereira -> Manizales (3 ciudades, 77 km)
Número de rutas: 2
Ruta más corta: Cartago -> Pereira -> Manizales
Ruta de menor distancia: Cartago -> Pereira -> Manizales (77 km)

=== Armenia -> Bogota ===
Sin rutas: el destino no es alcanzable desde el origen
Número de rutas: 0

=== Armenia -> Cali ===
No existe en el grafo: Cali

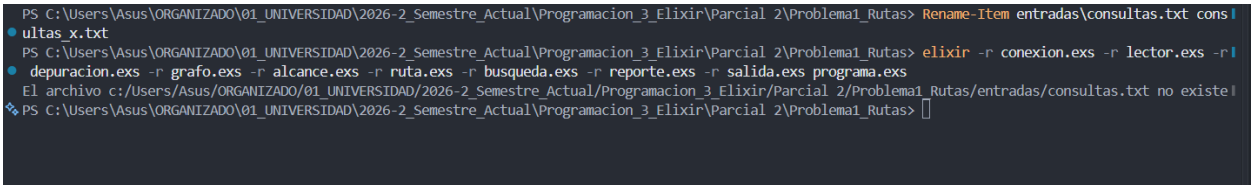

----- consultas ignoradas -----
Línea 6: Pereira -> consulta mal formada
PS C:\Users\Asus\ORGANIZADO\01_UNIVERSIDAD\2026-2_Semestre_Actual\Programacion_3_Elixir\Parcial 2\Problema1_Rutas>
```



2.9 Casos de error

Caso	Entrada	Resultado esperado	Estado
Línea vacía y comentario	Líneas 9 y 10 de conexiones.csv	Se ignoran	Verificado
Campos faltantes	Línea 11	Reporte en errores_conexiones.txt	Verificado
Distancia no numérica	Línea 12	Reporte en errores_conexiones.txt	Verificado
Destino inalcanzable	Consulta Armenia,Bogota	"Sin rutas" sin ejecutar la búsqueda	Verificado
Ciudad inexistente	Consulta Armenia,Cali	"No existe en el grafo: Cali"	Verificado
Consulta mal formada	Consulta Pereira	Se ignora y se informa	Verificado
Archivo inexistente	Renombrar consultas.txt	Mensaje claro, sin excepción	**Verificado**

Tabla 9. *Casos de error del problema 1.*



2.10 Estado de validación del problema 1

Requisito del enunciado	Estado
Búsqueda recursiva de todas las rutas sin repetir ciudades	Verificado
Número de ciudades y distancia total por ruta	Verificado
Número de rutas, rutas más cortas y ruta de menor distancia	Verificado
Varias consultas desde consultas.txt	Verificado
Structs %Conexion{} y %Ruta{}; Conexion.parse/1	Verificado
Líneas mal formadas en errores_conexiones.txt	Verificado
Depuración recursiva en depuracion_conexiones.txt	Verificado
Stream infinito de alcance en alcance.txt	Verificado
Ciudad inexistente en una consulta	Verificado
Archivo de entrada inexistente	**Verificado**
Capturas de pantalla de todas las evidencias	**Verificado**

Tabla 10. *Estado de los requisitos del problema 1.*

2.11 Bitácora de uso de IA (problema 1)

Registro de las interacciones con la herramienta de IA durante el desarrollo de este problema. Cada problema tiene su propia bitácora al final de su capítulo; la reflexión general sobre el uso de IA en todo el parcial está en el capítulo 4.

2.11.1 Interacciones registradas

Módulo	Qué se pidió a la IA	Qué se hizo con la respuesta
Organización	Revisar el material del curso y proponer una arquitectura acorde con lo enseñado	Se adoptó la arquitectura por módulos; se corrigió la ubicación de los archivos que estaban dentro de salidas/
`conexion.exs`	Explicar qué es un struct y cómo se define	El estudiante escribió el struct; primero intentó crear tres structs y luego entendió que es uno con tres campos
`conexion.exs`	Plantilla de `parse/1`	El estudiante completó los espacios; se corrigieron el separador del `split`, el patrón `{distancia, ""}` y los mensajes de error. El estudiante identificó que "Armenia, Pereira, 45" fallaba por los espacios y se agregó `Enum.map(&String.trim/1)`
Entorno	Error al ejecutar `iex` en PowerShell	Se aprendió que en PowerShell `iex` es un alias de Invoke-Expression y se usó `iex.bat`
`lector.exs`	Plantilla de lectura con Streams	Se completaron y corrigieron `ignorar?/1` y `clasificar/2`
`depuracion.exs`	Plantilla de la depuración recursiva	Se corrigieron 5 de 6 espacios (caso base con variable en lugar de `[]`, uso de `previa` en lugar de `resto`, entre otros) y se explicó cada error
`grafo.exs`	Plantilla con `Enum.reduce` y `Map.update`	Se corrigió el acumulador inicial (`[]` en lugar de `%{}`)
`alcance.exs`	Plantilla del Stream infinito	Se corrigieron el paso 0, el operador `=!` (que Elixir interpreta como asignación y negación) y el argumento de `List.last`
`busqueda.exs`	Plantilla de la búsqueda con backtracking	Se corrigieron la distancia inicial, la suma de kilómetros y la llamada con `resto`
Integración	Módulos de lectura de consultas, reporte, salida y programa principal	Fueron escritos por la IA por limitación de tiempo y luego ejecutados y verificados por el estudiante con casos normales y de error
Pruebas y evidencias	Comandos para cada caso de prueba y para tomar las capturas	Ejecuté cada prueba y tomé las capturas; la IA me ayudó con errores de entorno, como dos comandos pegados en una misma línea de iex y variables perdidas al abrir una sesión nueva
Documentación	Estructura del documento y redacción de la reflexión	La IA generó la estructura del documento y un borrador de la reflexión a partir de la sesión; yo pegué las capturas y revisé el contenido

Tabla 11. *Bitácora de uso de IA en el problema 1.*

Revisé cada fila de la bitácora y corresponde a lo que hice durante el desarrollo. Trabajé con plantillas que tenían espacios para completar: yo escribía mi respuesta, la IA revisaba el archivo directamente en mi carpeta y me explicaba qué estaba mal y por qué. En los últimos módulos, por el tiempo, la IA corrigió directamente los errores y yo verifiqué el resultado ejecutando las pruebas.

2.11.2 Un ejemplo concreto

La interacción que más me sirvió fue la construcción de `Conexion.parse/1`. Le pedí a la IA que me explicara cómo convertir una línea del CSV en un struct y me dio una plantilla con espacios para completar. En mi primer intento puse dentro de `String.split` una lista con los nombres de los campos y la coma; la IA me explicó que el separador solo indica dónde cortar el texto y que los nombres se asignan después, con el pattern matching `[origen, destino, distancia]`. Ahí entendí que `String.split` funciona como unas tijeras: corta por el carácter que uno le indique entre comillas.

Después, antes de ejecutar la prueba con la línea "Armenia, Pereira, 45", la IA me pidió predecir el resultado. Me di cuenta de que la línea pasaba la validación de tres campos, pero la distancia " 45" no se podía convertir a número por el espacio inicial, y que además la ciudad quedaría guardada como " Pereira". Con eso entendí que `String.trim` aplicado a la línea completa solo limpia los bordes y que había que limpiar cada campo con `Enum.map(&String.trim/1)`.

Cuando lo intenté escribir con una función anónima me equivoqué varias veces (escribí `Strin.trim`, le pasé el texto " " en vez del campo y olvidé el `end`). Además, al probar seguía saliendo el error porque iex tenía cargada la versión vieja del archivo y había que recompilar. Al final la prueba devolvió `{:ok, %Conexion{origen: "Armenia", destino: "Pereira", distancia: 45}}`. De esta interacción aprendí a predecir qué hace el código antes de ejecutarlo y a revisar no solo si el programa falla, sino si el dato que produce es correcto.

3. Problema 2: Escapar de la isla

Estado del capítulo: el problema 2 está completo. Todos los módulos están implementados; el programa se ejecutó con el mapa del enunciado, con el mapa inválido y con un archivo inexistente, y los archivos de salida coinciden con lo esperado (sección 3.7).

3.1 Enunciado resumido

Una persona atrapada en una isla debe llegar desde su posición inicial P hasta el bote B. El programa debe encontrar, mediante recursión, todas las rutas posibles moviéndose solo hacia arriba, abajo, izquierda o derecha y sin visitar dos veces la misma posición. Para cada ruta se muestran sus coordenadas, el número de movimientos y el costo total; al final se informa el total de rutas, la ruta más corta (menos movimientos) y la de menor costo.

Requisitos adicionales: los símbolos y los costos se leen de `terrenos.csv` (no pueden estar escritos en el código); se definen los structs `%Terreno{}` y `%Ruta{}`; las posiciones no transitables se descartan con pattern matching sobre el struct; y antes de buscar, el mapa se valida con recursión propia (filas del mismo ancho, símbolos definidos, exactamente una P y un B, borde de agua).

3.2 Arquitectura

Módulo	Responsabilidad	Estado
`Terreno`	Struct y conversión de una línea de terrenos.csv	Implementado y probado
`Ruta`	Struct de una ruta, movimientos, recorrido y línea CSV	Implementado y probado
`Lector`	Lectura perezosa de terrenos.csv e isla.txt	Implementado y probado
`Validacion`	Validación recursiva del mapa (sin Enum ni Stream)	Implementado y probado
`Mapa`	Isla como mapa {fila, columna} => %Terreno{}, ubicación de símbolos y vecinos	Implementado y probado
`Busqueda`	Todas las rutas con recursión y backtracking	Implementado y probado
`Reporte`, `Salida`, `Programa`	Resumen, archivos de salida y función main	**Implementado y probado**

Tabla 12. *Módulos del problema 2 y su estado actual.*

```
terrenos.csv ─File.stream!→ Terreno.parse → %{"C" => %Terreno{...}, "T" =>
%Terreno{transitable: false}, ...}
isla.txt ────File.stream!→ String.split ──→ [["~","~",...], ["~","P","C",...], ...]
                    │
        Validacion.validar (recursión propia) ── inválido ──→ validacion_isla.txt
(pendiente)
                    │ válido
        Mapa.construir ──→ %{{0,0} => %Terreno{...}, {1,1} => %Terreno{simbolo:
"P"}, ...}
        Mapa.buscar "P" y "B" ──→ inicio {1,1}, bote {5,5}
                    │
        Busqueda.rutas (recursión + backtracking) ──→ [%Ruta{posiciones, costo}, ...]
                    │
        Reporte ──→ rutas_isla.csv, resumen_isla.txt con el mapa marcado con *
(pendiente)
```

Figura 25. *Recorrido de la información en el problema 2.*

El diseño reutiliza el del problema 1: la búsqueda tiene la misma estructura de dos funciones (`buscar` y `explorar`) y la misma técnica de caso base con variable repetida. Cambian la forma de obtener los vecinos (las cuatro direcciones) y la condición para entrar a una posición, que ahora depende del terreno.

3.3 Datos de entrada

Se usaron los datos del ejemplo del enunciado. Las coordenadas se escriben como `{fila, columna}` y empiezan en 0.

```
        col: 0 1 2 3 4 5 6
fila 0:      ~ ~ ~ ~ ~ ~ ~
fila 1:      ~ P C C C C ~      P (inicio) en {1,1}
fila 2:      ~ R T T T C ~
fila 3:      ~ D C T C C ~
fila 4:      ~ R T C A T ~
fila 5:      ~ C D C C B ~      B (bote) en {5,5}
fila 6:      ~ ~ ~ ~ ~ ~ ~
```

Figura 26. *Archivo entradas/isla.txt con la numeración de filas y columnas.*

```
# simbolo,nombre,transitable,costo
C,camino,si,1
A,arena,si,1
D,terreno difícil,si,2
R,terreno rocoso,si,3
T,árbol,no,0
~,agua,no,0
P,inicio,si,0
B,bote,si,1
```

Figura 27. *Archivo entradas/terrenos.csv.*

```
# mapa con errores a proposito para probar la validacion
~ ~ ~ ~ C ~ ~
~ P C C C C ~
~ R X T T C ~
~ D C T C ~
~ R T P A T ~
~ C D C C B ~
~ ~ ~ ~ ~ ~ ~
```

Figura 28. *Archivo entradas/isla_invalida.txt, construido para probar la validación.*

Fila (sin contar el comentario)	Error introducido	Regla que debe detectarlo
0	C en la posición {0,4}	El borde debe ser agua
2	X en la posición {2,2}	Todos los símbolos deben estar en terrenos.csv

Fila (sin contar el comentario)	Error introducido	Regla que debe detectarlo
3	Solo 6 posiciones	Todas las filas deben tener el mismo número de posiciones
4	Segunda P en {4,3}	Exactamente una posición inicial P

Tabla 13. *Errores incluidos a propósito en isla_invalida.txt.*

3.4 Desarrollo por módulo

3.4.1 Terreno (terreno.exs)

Define `%Terreno{simbolo, nombre, transitable, costo}` con los cuatro campos obligatorios. `parse/1` sigue el mismo esquema de `Conexion.parse/1`: limpia la línea, la separa por comas, limpia cada campo y verifica con pattern matching que haya cuatro campos.

La conversión usa `with` para encadenar dos validaciones: `convertir_transitable/1` transforma el texto `"si"`/`"no"` en el booleano `true`/`false` (cualquier otro valor es un error) e `Integer.parse/1` debe devolver `{valor, ""}`. Convertir `transitable` a booleano es lo que permite después descartar posiciones con el patrón `%Terreno{transitable: false}`.

```
defmodule Terreno do
  @moduledoc """
  Representa un tipo de terreno de la isla: su símbolo, nombre, si se puede
  transitar y el costo de entrar a una posición de ese tipo.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  @enforce_keys [:simbolo, :nombre, :transitable, :costo]
  defstruct [:simbolo, :nombre, :transitable, :costo]

  def parse(linea) do
    campos =
      linea
      |> String.trim()
      |> String.split(",")
      |> Enum.map(&String.trim/1)

    case campos do
      [simbolo, nombre, transitable, costo] ->
        convertir(simbolo, nombre, transitable, costo)

      _ ->
        {:error, "campos faltantes"}
    end
  end

  defp convertir(simbolo, nombre, transitable, costo) do
    with {:ok, es_transitable} <- convertir_transitable(transitable),
         {valor, ""} <- Integer.parse(costo) do
      {:ok, %Terreno{simbolo: simbolo, nombre: nombre, transitable: es_transitable,
costo: valor}}
    else
      {:error, motivo} -> {:error, motivo}
      _ -> {:error, "costo no numérico"}
    end
  end
```

```
  defp convertir_transitable("si"), do: {:ok, true}
  defp convertir_transitable("no"), do: {:ok, false}
  defp convertir_transitable(_), do: {:error, "transitable debe ser si o no"}
end
```

Figura 29. *Código fuente de terreno.exs.*

Evidencia de ejecución:



Figura 30. *Pruebas de Terreno.parse/1 en iex: dos líneas válidas y tres inválidas (campos faltantes, transitable inválido y costo no numérico).*

3.4.2 Ruta (ruta.exs)

Define `%Ruta{posiciones, costo}`. El número de movimientos se calcula a partir de las posiciones, como exige el enunciado: `length(posiciones) - 1`, porque la primera posición es el punto de partida y no un movimiento.

`posicion_texto/1` escribe cada coordenada como `{fila,columna}` sin espacio, igual que el enunciado, y `a_csv/2` genera la línea de `rutas_isla.csv` separando los campos con `;`, porque las coordenadas contienen comas.

```
defmodule Ruta do
  @moduledoc """
  Representa una ruta en la isla: la lista de posiciones {fila, columna}
  en orden y el costo total de la ruta.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  @enforce_keys [:posiciones, :costo]
```

```
  defstruct [:posiciones, :costo]

  def movimientos(%Ruta{posiciones: posiciones}), do: length(posiciones) - 1

  def posicion_texto({fila, columna}), do: "{#{fila},#{columna}}"

  def recorrido(%Ruta{posiciones: posiciones}) do
    posiciones
    |> Enum.map(&posicion_texto/1)
    |> Enum.join(" -> ")
  end

  def a_csv(%Ruta{} = ruta, numero) do
    "#{numero};#{movimientos(ruta)};#{ruta.costo};#{recorrido(ruta)}"
  end
end
```

Figura 31. *Código fuente de ruta.exs.*

Evidencia de ejecución:

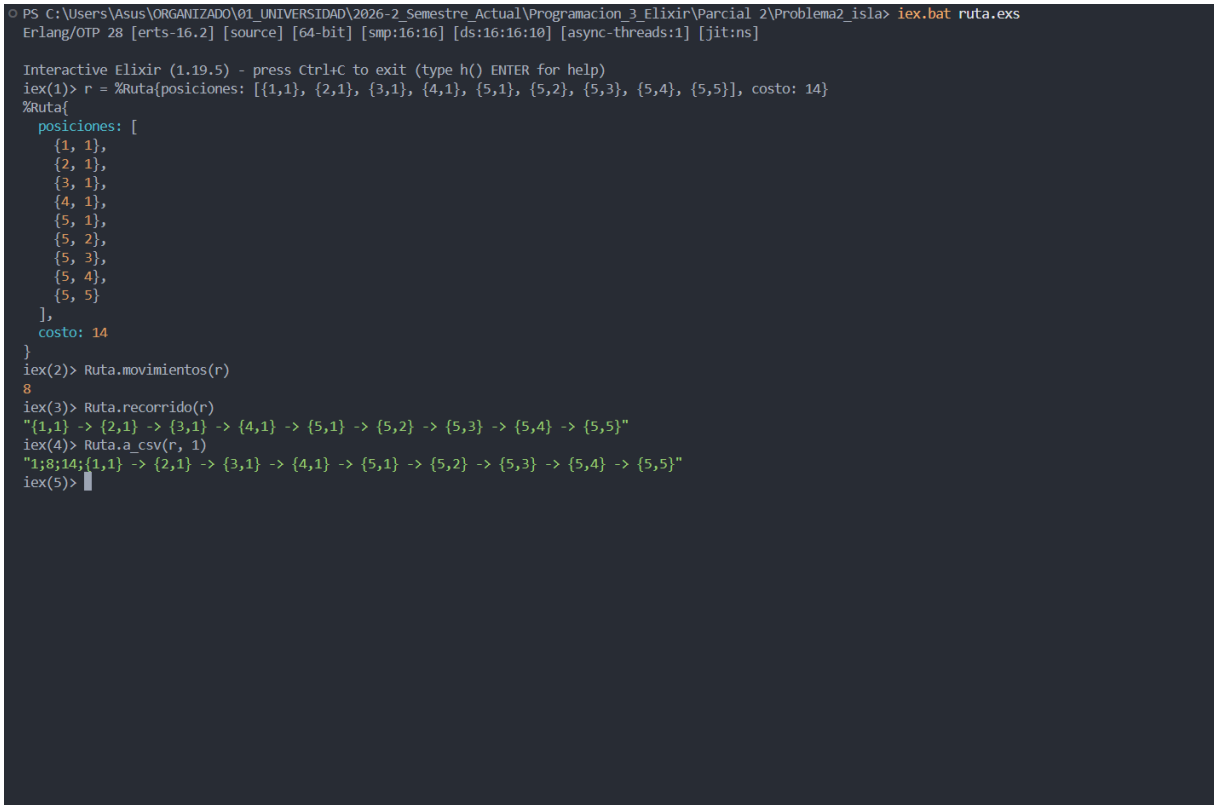


Figura 32. *Pruebas de Ruta en iex: movimientos, recorrido y línea CSV de la ruta de ejemplo del enunciado (8 movimientos, costo 14).*

3.4.3 Lector (lector.exs)

Reutiliza la función general `leer/2` del problema 1: `File.stream!`, numeración de líneas, descarte perezoso de líneas vacías y comentarios y un único `Enum.reduce` al final. `leer_terrenos/1` convierte las líneas válidas en un mapa `símbolo => %Terreno{}` con `Map.new/2`, de modo que la búsqueda

consulta el terreno de un símbolo directamente. `leer_isla/1` separa cada fila por espacios con `String.split/1` y devuelve la lista de filas.

Si el archivo no existe, ambas funciones devuelven `{:error, mensaje}` sin lanzar una excepción.

```
defmodule Lector do
  @moduledoc """
  Lee los archivos de entrada de la isla de forma perezosa, línea por línea.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def leer_terrenos(ruta) do
    with {:ok, validos, errores} <- leer(ruta, &Terreno.parse/1) do
      terrenos = Map.new(validos, fn {_numero, _linea, terreno} -> {terreno.simbolo,
terreno} end)
      {:ok, terrenos, errores}
    end
  end

  def leer_isla(ruta) do
    with {:ok, filas, _errores} <- leer(ruta, fn linea -> {:ok, String.split(linea)} end)
do
      {:ok, Enum.map(filas, fn {_numero, _linea, simbolos} -> simbolos end)}
    end
  end

  defp leer(ruta, convertir) do
    if File.exists?(ruta) do
      {validas, errores} =
        ruta
        |> File.stream!()
        |> Stream.with_index(1)
        |> Stream.map(fn {linea, numero} -> {numero, String.trim(linea)} end)
        |> Stream.reject(fn {_numero, linea} -> ignorar?(linea) end)
        |> Stream.map(fn {numero, linea} -> {numero, linea, convertir.(linea)} end)
        |> Enum.reduce({[], []}, &clasificar/2)

      {:ok, Enum.reverse(validas), Enum.reverse(errores)}
    else
      {:error, "El archivo #{ruta} no existe"}
    end
  end

  defp ignorar?(""), do: true
  defp ignorar?(linea), do: String.starts_with?(linea, "#")

  defp clasificar({numero, linea, {:ok, dato}}, {validas, errores}) do
    {[{numero, linea, dato} | validas], errores}
  end

  defp clasificar({numero, linea, {:error, motivo}}, {validas, errores}) do
    {validas, [{numero, linea, motivo} | errores]}
  end
end
```

Figura 33. *Código fuente de lector.exs.*

Evidencia de ejecución:



Figura 34. *Lectura de terrenos.csv (8 terrenos, sin errores), de isla.txt (7 filas de 7 símbolos) y de un archivo inexistente.*

3.4.4 Validacion (validacion.exs)

Implementa las cuatro reglas del mapa con recursión propia, sin `Enum` ni `Stream`, y acumula todos los problemas en una lista en lugar de detenerse en el primero. Su funcionamiento se detalla en la sección 3.5.

```
defmodule Validacion do
  @moduledoc """
  Valida el mapa de la isla antes de buscar rutas, usando solo recursión
  propia (sin Enum ni Stream):
  - todas las filas tienen el mismo número de posiciones;
  - todos los símbolos aparecen en terrenos.csv;
  - hay exactamente una posición inicial P y exactamente un bote B;
  - todo el borde del mapa es agua (~).
  Devuelve :ok o {:error, problemas} con TODOS los problemas encontrados.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def validar(filas, terrenos) do
    total = length(filas)
    ancho = ancho_esperado(filas)

    problemas = revisar_anchos(filas, 0, ancho, [])
    problemas = revisar_filas(filas, 0, total, terrenos, problemas)
    problemas = revisar_cantidad(posiciones_de(filas, "P", 0, []), "posiciones iniciales
P", "No hay posición inicial P", problemas)
    problemas = revisar_cantidad(posiciones_de(filas, "B", 0, []), "botes B", "No hay
bote B", problemas)
```

```
    case invertir(problemas, []) do
      [] -> :ok
      lista -> {:error, lista}
    end
  end

  defp ancho_esperado([]), do: 0
  defp ancho_esperado([primera | _resto]), do: length(primera)

  defp revisar_anchos([], _f, _ancho, problemas), do: problemas

  defp revisar_anchos([fila | resto], f, ancho, problemas) do
    if length(fila) == ancho do
      revisar_anchos(resto, f + 1, ancho, problemas)
    else
      problema = "Fila #{f}: tiene #{length(fila)} posiciones; se esperaban #{ancho}"
      revisar_anchos(resto, f + 1, ancho, [problema | problemas])
    end
  end

  defp revisar_filas([], _f, _total, _terrenos, problemas), do: problemas

  defp revisar_filas([fila | resto], f, total, terrenos, problemas) do
    problemas = revisar_columnas(fila, f, 0, length(fila) - 1, total, terrenos,
problemas)
    revisar_filas(resto, f + 1, total, terrenos, problemas)
  end

  defp revisar_columnas([], _f, _c, _ultima, _total, _terrenos, problemas), do: problemas

  defp revisar_columnas([simbolo | resto], f, c, ultima, total, terrenos, problemas) do
    problemas = revisar_simbolo(simbolo, f, c, terrenos, problemas)
    problemas = revisar_borde(simbolo, f, c, ultima, total, problemas)
    revisar_columnas(resto, f, c + 1, ultima, total, terrenos, problemas)
  end

  defp revisar_simbolo(simbolo, f, c, terrenos, problemas) do
    if Map.has_key?(terrenos, simbolo) do
      problemas
    else
      ["Posición #{posicion(f, c)}: el símbolo #{simbolo} no está en terrenos.csv" |
problemas]
    end
  end

  defp revisar_borde("~", _f, _c, _ultima, _total, problemas), do: problemas

  defp revisar_borde(simbolo, f, c, ultima, total, problemas) do
    if f == 0 or f == total - 1 or c == 0 or c == ultima do
      ["Posición #{posicion(f, c)}: el borde debe ser agua y contiene #{simbolo}" |
problemas]
    else
      problemas
    end
  end

  defp posiciones_de([], _simbolo, _f, encontradas), do: invertir(encontradas, [])

  defp posiciones_de([fila | resto], simbolo, f, encontradas) do
    posiciones_de(resto, simbolo, f + 1, buscar_en_fila(fila, simbolo, f, 0,
encontradas))
  end

  defp buscar_en_fila([], _simbolo, _f, _c, encontradas), do: encontradas
```

```
  defp buscar_en_fila([simbolo | resto], simbolo, f, c, encontradas) do
    buscar_en_fila(resto, simbolo, f, c + 1, [{f, c} | encontradas])
  end

  defp buscar_en_fila([_otro | resto], simbolo, f, c, encontradas) do
    buscar_en_fila(resto, simbolo, f, c + 1, encontradas)
  end

  defp revisar_cantidad([_una], _plural, _faltante, problemas), do: problemas
  defp revisar_cantidad([], _plural, faltante, problemas), do: [faltante | problemas]

  defp revisar_cantidad(varias, plural, _faltante, problemas) do
    ["Hay #{length(varias)} #{plural}: #{unir(varias)}" | problemas]
  end

  defp unir([{f, c}]), do: posicion(f, c)
  defp unir([{f1, c1}, {f2, c2}]), do: posicion(f1, c1) <> " y " <> posicion(f2, c2)
  defp unir([{f, c} | resto]), do: posicion(f, c) <> ", " <> unir(resto)

  defp posicion(f, c), do: "{#{f},#{c}}"

  defp invertir([], acumulado), do: acumulado
  defp invertir([cabeza | cola], acumulado), do: invertir(cola, [cabeza | acumulado])
end
```

Figura 35. *Código fuente de validacion.exs.*

Evidencia de ejecución:

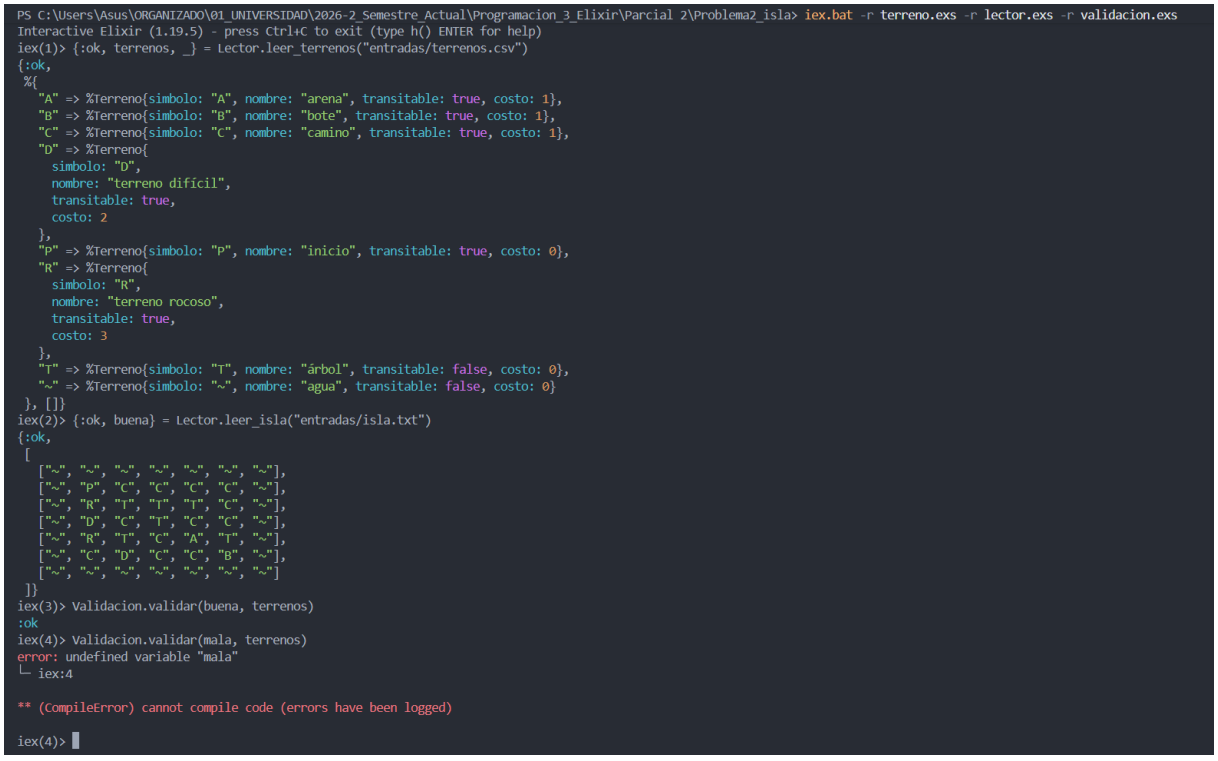


Figura 36. *Validación en iex: el mapa del enunciado devuelve :ok. La segunda llamada falló con "undefined variable mala" porque no se ejecutó antes la lectura de isla_invalida.txt; no es un error del módulo.*

Validación del mapa inválido y del archivo inexistente, ejecutadas con el programa completo (sección 3.7):

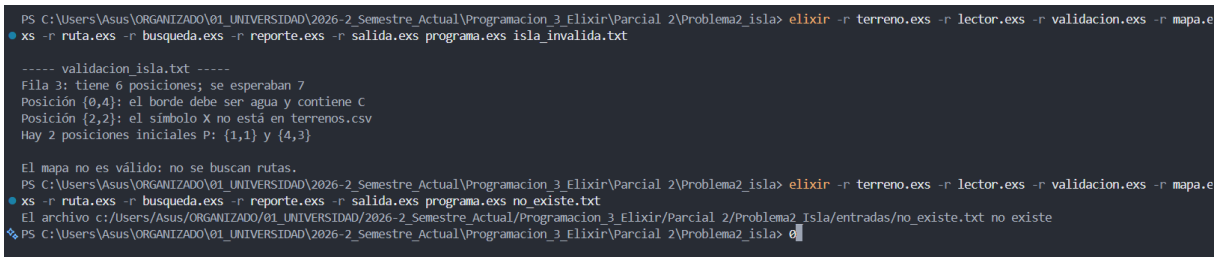


Figura 37. *Ejecución con isla_invalida.txt (4 problemas reportados y búsqueda omitida) y con un archivo inexistente (mensaje claro, sin excepción).*

3.4.5 Mapa (mapa.exs)

`construir/2` convierte la lista de filas en un mapa `{fila, columna} => %Terreno{}` usando `Enum.with_index` para numerar filas y columnas. Con esta representación, consultar una posición es directo con `Map.get/2`, y una posición fuera de la isla devuelve `nil` sin causar errores.

`buscar/2` devuelve la coordenada de un símbolo (se usa para ubicar `P` y `B`), `terreno_en/2` el terreno de una posición y `vecinos/1` las cuatro posiciones adyacentes: arriba `{f - 1, c}`, abajo `{f + 1, c}`, izquierda `{f, c - 1}` y derecha `{f, c + 1}`.

```
defmodule Mapa do
  @moduledoc """
  Representa la isla como un mapa {fila, columna} => %Terreno{}, ubica
  símbolos y calcula las posiciones vecinas (arriba, abajo, izquierda, derecha).
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def construir(filas, terrenos) do
    filas
    |> Enum.with_index()
    |> Enum.flat_map(fn {fila, f} ->
      fila
      |> Enum.with_index()
      |> Enum.map(fn {simbolo, c} -> {{f, c}, Map.get(terrenos, simbolo)} end)
    end)
    |> Map.new()
  end

  def buscar(mapa, simbolo) do
    {posicion, _terreno} = Enum.find(mapa, fn {_posicion, terreno} -> terreno.simbolo ==
simbolo end)
    posicion
  end

  def terreno_en(mapa, posicion), do: Map.get(mapa, posicion)

  def vecinos({f, c}) do
    [
      {f - 1, c},
      {f + 1, c},
      {f, c - 1},
      {f, c + 1}
    ]
  end
end
```

Figura 38. *Código fuente de mapa.exs.*

Evidencia de ejecución:



Figura 39. *Carga de terrenos e isla en la sesión de iex usada para probar el mapa y la búsqueda.*



Figura 40. *Inicio del resultado de Mapa.construir/2: cada coordenada asociada a su %Terreno{}.*

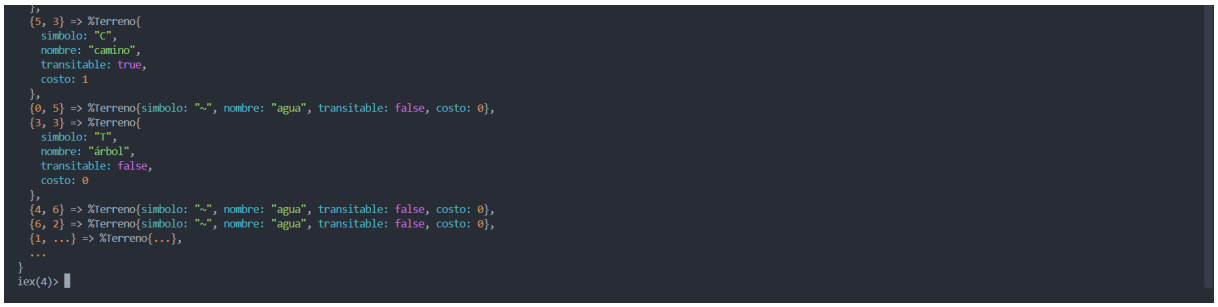


Figura 41. *Final del resultado de Mapa.construir/2 (iex abrevia el resto con …).*



Figura 42. *Ubicación de la posición inicial P en {1, 1} y del bote B en {5, 5} con Mapa.buscar/2.*

3.4.6 Busqueda (busqueda.exs)

Encuentra todas las rutas con recursión y backtracking. La decisión de entrar o no a una posición se toma en `intentar/6`, que tiene tres cláusulas: posición fuera del mapa (`nil`), posición no transitable (patrón `%Terreno{transitable: false}`, como exige el enunciado) y posición transitable, donde además se comprueba que no esté ya en el camino. Su funcionamiento se detalla en la sección 3.6.

```
defmodule Busqueda do
  @moduledoc """
  Encuentra todas las rutas desde la posición inicial hasta el bote con
  recursión y backtracking. Solo se mueve arriba, abajo, izquierda o derecha,
  no repite posiciones en una misma ruta y descarta con pattern matching las
  posiciones no transitables.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def rutas(mapa, inicio, bote), do: buscar(mapa, inicio, bote, [inicio], 0)

  defp buscar(_mapa, bote, bote, camino, costo) do
    [%Ruta{posiciones: camino, costo: costo}]
  end

  defp buscar(mapa, actual, bote, camino, costo) do
    explorar(Mapa.vecinos(actual), mapa, bote, camino, costo)
  end

  defp explorar([], _mapa, _bote, _camino, _costo), do: []

  defp explorar([posicion | resto], mapa, bote, camino, costo) do
    rutas_por_vecino = intentar(Mapa.terreno_en(mapa, posicion), posicion, mapa, bote,
camino, costo)
    rutas_por_vecino ++ explorar(resto, mapa, bote, camino, costo)
  end

  defp intentar(nil, _posicion, _mapa, _bote, _camino, _costo), do: []

  defp intentar(%Terreno{transitable: false}, _posicion, _mapa, _bote, _camino, _costo),
do: []

  defp intentar(%Terreno{costo: costo_casilla}, posicion, mapa, bote, camino, costo) do
    if posicion in camino do
      []
    else
```

```
      buscar(mapa, posicion, bote, camino ++ [posicion], costo + costo_casilla)
    end
  end
end
```

Figura 43. *Código fuente de busqueda.exs.*

Evidencia de ejecución:

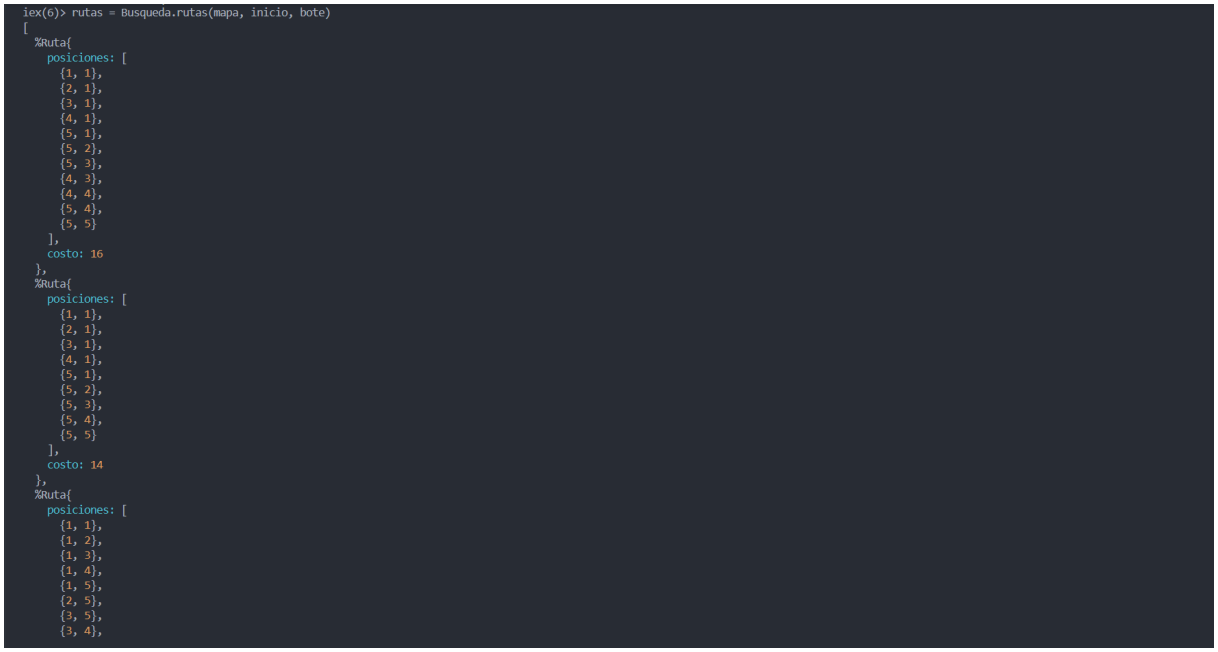


Figura 44. *Inicio del resultado de Busqueda.rutas/3: lista de %Ruta{} con sus posiciones y su costo.*

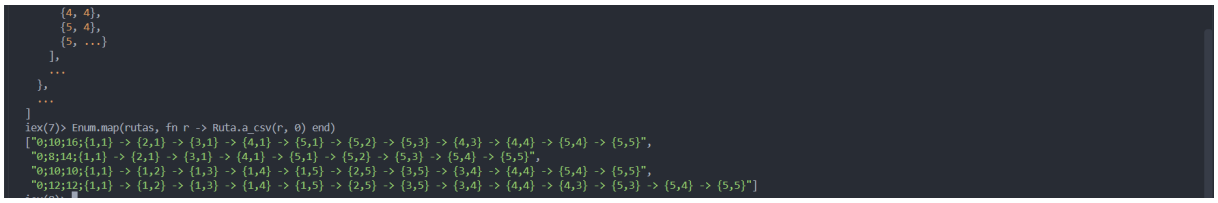


Figura 45. *Las 4 rutas encontradas en formato CSV (número;movimientos;costo;recorrido). El número aparece como 0 porque en la prueba se pasó 0 a Ruta.a_csv/2.*

3.4.7 Reporte, Salida y Programa

Implementan la integración final. `Reporte` construye las líneas de texto sin imprimir ni escribir: `lineas_rutas/1` (cada ruta con sus coordenadas, movimientos y costo), `lineas_csv/1` (encabezado `ruta;movimientos;costo;recorrido` y una línea por ruta), `lineas_resumen/2` (total, rutas más cortas —todas en caso de empate—, ruta de menor costo y el mapa marcado) y `mapa_marcado/2`, que reemplaza por `*` las posiciones intermedias de la ruta de menor costo. Se dejan visibles la posición inicial y el bote; así el módulo no necesita tener escritos los símbolos P ni B, porque usa la primera y la última posición de la ruta.

`Salida` es idéntico al del problema 1: escribe línea por línea con `Enum.into(File.stream!(ruta, [:write]))`. `Programa` lee ambos archivos con `with` (un archivo inexistente produce un mensaje y

termina), valida el mapa y, solo si es válido, construye el mapa, busca las rutas y genera los archivos. Recibe opcionalmente el nombre del mapa como argumento (`System.argv()`), lo que permite probar `isla_invalida.txt` sin renombrar archivos.

```
defmodule Reporte do
  @moduledoc """
  Construye las líneas de texto de la pantalla y de los archivos de salida.
  No escribe ni imprime nada: solo transforma datos en texto.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def lineas_rutas(rutas) do
    rutas
    |> Enum.with_index(1)
    |> Enum.flat_map(fn {ruta, numero} ->
      [
        "Ruta #{numero}:",
        Ruta.recorrido(ruta),
        "Número de movimientos: #{Ruta.movimientos(ruta)}",
        "Costo: #{ruta.costo}",
        ""
      ]
    end)
  end

  def lineas_csv(rutas) do
    filas =
      rutas
      |> Enum.with_index(1)
      |> Enum.map(fn {ruta, numero} -> Ruta.a_csv(ruta, numero) end)

    ["ruta;movimientos;costo;recorrido" | filas]
  end

  def lineas_resumen([], _filas) do
    ["Número total de rutas: 0", "No existe ninguna ruta desde la posición inicial hasta
el bote"]
  end

  def lineas_resumen(rutas, filas) do
    cortas = mas_cortas(rutas)
    barata = menor_costo(rutas)

    ["Número total de rutas: #{length(rutas)}",
     "Ruta más corta: #{Ruta.movimientos(hd(cortas))} movimientos"] ++
      Enum.map(cortas, fn ruta -> "  " <> Ruta.recorrido(ruta) end) ++
      ["Ruta de menor costo: costo #{barata.costo} (#{Ruta.movimientos(barata)}
movimientos)",
       "  " <> Ruta.recorrido(barata),
       "",
       "Mapa con la ruta de menor costo marcada con *:"] ++
      mapa_marcado(filas, barata)
  end

  def mas_cortas(rutas) do
    minimo = rutas |> Enum.map(&Ruta.movimientos/1) |> Enum.min()
    Enum.filter(rutas, fn ruta -> Ruta.movimientos(ruta) == minimo end)
  end

  def menor_costo(rutas), do: Enum.min_by(rutas, fn ruta -> ruta.costo end)

  def mapa_marcado(filas, %Ruta{posiciones: posiciones}) do
```

```
    recorridas = Enum.slice(posiciones, 1..-2//1)

    filas
    |> Enum.with_index()
    |> Enum.map(fn {fila, f} ->
      fila
      |> Enum.with_index()
      |> Enum.map(fn {simbolo, c} -> marcar(simbolo, {f, c} in recorridas) end)
      |> Enum.join(" ")
    end)
  end

  defp marcar(_simbolo, true), do: "*"
  defp marcar(simbolo, false), do: simbolo
end
```

Figura 46. *Código fuente de reporte.exs.*

```
defmodule Salida do
  @moduledoc """
  Escribe resultados en archivos línea por línea y los muestra en pantalla.
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  def escribir_lineas(lineas, ruta) do
    lineas
    |> Stream.map(fn linea -> linea <> "\n" end)
    |> Enum.into(File.stream!(ruta, [:write]))

    :ok
  end

  def mostrar_lineas(lineas), do: Enum.each(lineas, &IO.puts/1)
end
```

Figura 47. *Código fuente de salida.exs.*

```
defmodule Programa do
  @moduledoc """
  Problema 2: escapar de la isla. Orquesta la lectura, la validación del
  mapa, la búsqueda de rutas y la escritura de resultados.
  Uso: elixir -r ... programa.exs [archivo_isla]   (por defecto isla.txt)
  - Autor: César Leandro Ortegón
  - Fecha: Noviembre 2026
  """

  @entradas Path.join(__DIR__, "entradas")
  @salidas Path.join(__DIR__, "salidas")

  def main(argumentos) do
    archivo_isla = archivo_isla(argumentos)

    with {:ok, terrenos, errores_terrenos} <- Lector.leer_terrenos(Path.join(@entradas,
"terrenos.csv")),
         {:ok, filas} <- Lector.leer_isla(Path.join(@entradas, archivo_isla)) do
      File.mkdir_p!(@salidas)
      mostrar_errores_terrenos(errores_terrenos)

      case Validacion.validar(filas, terrenos) do
        :ok ->
```

```
          publicar(["El mapa es válido"], "validacion_isla.txt")
          resolver(filas, terrenos)

        {:error, problemas} ->
          publicar(problemas, "validacion_isla.txt")
          IO.puts("\nEl mapa no es válido: no se buscan rutas.")
      end
    else
      {:error, mensaje} -> IO.puts(:standard_error, mensaje)
    end
  end

  defp archivo_isla([]), do: "isla.txt"
  defp archivo_isla([archivo | _resto]), do: archivo

  defp resolver(filas, terrenos) do
    mapa = Mapa.construir(filas, terrenos)
    rutas = Busqueda.rutas(mapa, Mapa.buscar(mapa, "P"), Mapa.buscar(mapa, "B"))

    IO.puts("\n----- rutas encontradas -----")
    Salida.mostrar_lineas(Reporte.lineas_rutas(rutas))

    Salida.escribir_lineas(Reporte.lineas_csv(rutas), Path.join(@salidas,
"rutas_isla.csv"))
    IO.puts("Archivo generado: salidas/rutas_isla.csv")

    publicar(Reporte.lineas_resumen(rutas, filas), "resumen_isla.txt")
  end

  defp publicar(lineas, nombre) do
    Salida.escribir_lineas(lineas, Path.join(@salidas, nombre))
    IO.puts("\n----- #{nombre} -----")
    Salida.mostrar_lineas(lineas)
  end

  defp mostrar_errores_terrenos([]), do: :ok

  defp mostrar_errores_terrenos(errores) do
    IO.puts("----- líneas ignoradas de terrenos.csv -----")
    Enum.each(errores, fn {numero, linea, motivo} -> IO.puts("Línea #{numero}: #{linea} -
> #{motivo}") end)
  end
end

Programa.main(System.argv())
```

Figura 48. *Código fuente de programa.exs.*

3.5 Validación con recursión: el mapa de la isla

3.5.1 Funcionamiento

La validación recorre la matriz en tres niveles de recursión. `revisar_anchos/4` recorre las filas y compara la longitud de cada una con la de la fila 0. `revisar_filas/5` recorre las filas y, para cada una, `revisar_columnas/7` recorre sus posiciones aplicando `revisar_simbolo/5` (el símbolo debe existir en el mapa de terrenos) y `revisar_borde/6` (si la posición está en la primera o última fila, o en la primera o última columna, debe ser agua). Para contar `P` y `B`, `posiciones_de/4` recorre las filas y `buscar_en_fila/5` recoge las coordenadas donde la casilla coincide con el símbolo buscado, usando la

técnica de la variable repetida en el patrón. Finalmente `revisar_cantidad/4` distingue con pattern matching entre exactamente una coincidencia (`[_una]`), ninguna (`[]`) o varias.

Cada función recibe la lista de problemas acumulada y devuelve la lista ampliada, por lo que se reportan todos los problemas. Si la lista final está vacía, el resultado es `:ok`; si no, `{:error, problemas}` y la búsqueda no se ejecuta.

Función	Recorre	Caso base	Qué cambia en cada llamada
`revisar_anchos/4`	Filas	`[]`: devuelve los problemas	Se avanza a `resto` y `f + 1`
`revisar_filas/5`	Filas	`[]`: devuelve los problemas	Se avanza a `resto` y `f + 1`
`revisar_columnas/7`	Posiciones de una fila	`[]`: devuelve los problemas	Se avanza a `resto` y `c + 1`
`posiciones_de/4`	Filas	`[]`: devuelve las coordenadas encontradas	Se avanza a `resto` y `f + 1`
`buscar_en_fila/5`	Posiciones de una fila	`[]`: devuelve las coordenadas	Se avanza a `resto` y `c + 1`; si coincide, se agrega `{f, c}`
`invertir/2`	Lista de problemas	`[]`: devuelve el acumulado	La cabeza pasa al acumulado

Tabla 14. *Funciones recursivas de la validación del mapa.*

3.5.2 Prueba de escritorio con isla_invalida.txt

La prueba de escritorio se hizo a mano a partir del código y fue confirmada por la ejecución real: `validacion_isla.txt` contiene exactamente estos cuatro problemas, en el mismo orden (sección 3.7).

Paso	Función	Qué revisa	Problema anotado
1	`revisar_anchos`	Ancho de la fila 0 = 7; filas 1 a 6	Fila 3: tiene 6 posiciones; se esperaban 7
2	`revisar_columnas` (fila 0)	Posición {0,4} contiene C y está en el borde	Posición {0,4}: el borde debe ser agua y contiene C
3	`revisar_columnas` (fila 2)	Posición {2,2} contiene X	Posición {2,2}: el símbolo X no está en terrenos.csv
4	`revisar_columnas` (fila 3)	Última columna de la fila 3 es la 5 y contiene ~	(ninguno)
5	`posiciones_de "P"`	Encuentra {1,1} y {4,3}	Hay 2 posiciones iniciales P: {1,1} y {4,3}
6	`posiciones_de "B"`	Encuentra solo {5,5}	(ninguno: encaja con `[_una]`)
7	`invertir`	Ordena los problemas en el orden en que se encontraron	Resultado: `{:error, [4 problemas]}`

Tabla 15. *Prueba de escritorio de Validacion.validar/2 con el mapa inválido.*

3.5.3 Árbol de llamadas recursivas (fila 3 del mapa inválido)

```
revisar_filas([fila3, fila4, fila5, fila6], f=3, total=7, ...)
├─ revisar_columnas(["~","D","C","T","C","~"], f=3, c=0, ultima=5)
│   │ "~" en {3,0}: símbolo existe; borde (c == 0) pero es agua → sin problema
│   └─ revisar_columnas(["D","C","T","C","~"], 3, c=1, 5)
│       │ "D" en {3,1}: existe; no es borde
│       └─ revisar_columnas(["C","T","C","~"], 3, c=2, 5)
│           └─ revisar_columnas(["T","C","~"], 3, c=3, 5)
│               └─ revisar_columnas(["C","~"], 3, c=4, 5)
```

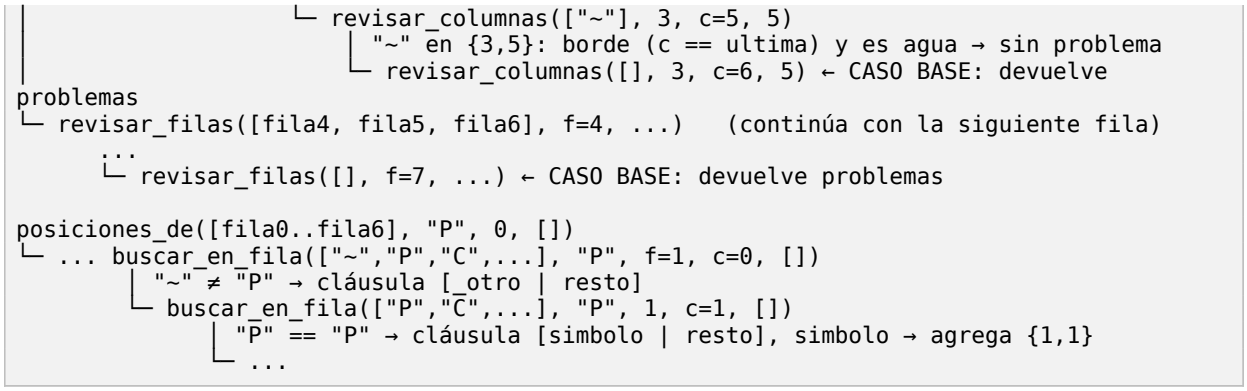


Figura 49. *Árbol de llamadas de la validación: niveles de filas y columnas, y búsqueda de P.*

3.6 Búsqueda principal: todas las rutas

3.6.1 Funcionamiento

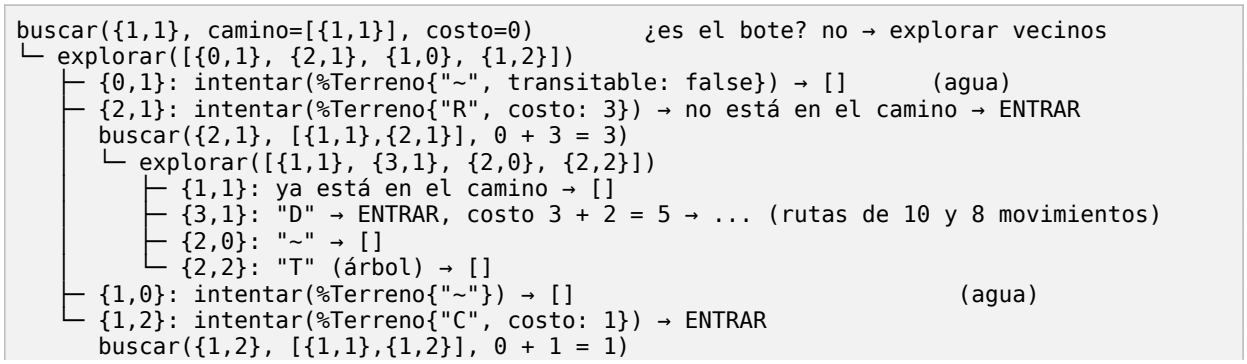
`Busqueda.rutas/3` inicia con el camino `[inicio]` y costo 0, porque la posición inicial no suma. `buscar/5` tiene como caso base la llegada al bote (patrón `buscar(_mapa, bote, bote, ...)`), donde devuelve una lista con un `%Ruta{}`. En otro caso pasa las cuatro posiciones vecinas a `explorar/5`, que las recorre con recursión propia: para cada vecino llama a `intentar/6` y luego se llama a sí misma con `resto`.

`intentar/6` decide con pattern matching: si el terreno es `nil` (fuera de la isla) o `%Terreno{transitable: false}` (árbol o agua) devuelve `[]`; si es transitable y la posición no está en el camino, entra con una llamada a `buscar` con el camino ampliado y el costo aumentado en el costo de esa casilla. El backtracking ocurre al continuar `explorar` con el camino y el costo originales.

Función	Caso base	Caso recursivo
`buscar/5`	Posición actual == bote → `[%Ruta{}]`	Llama a `explorar` con los 4 vecinos
`explorar/5`	`[]` → `[]`	Prueba el primer vecino con `intentar` y se llama con `resto`
`intentar/6`	`nil` o `%Terreno{transitable: false}` → `[]`	Si la posición no está en el camino, llama a `buscar`

Tabla 16. *Estructura recursiva de la búsqueda de rutas en la isla.*

3.6.2 Árbol de llamadas (primeros niveles)



```
    └─ ... (rutas de 10 y 12 movimientos por la fila 1 y la columna 5)
```

Figura 50. *Primeros niveles del árbol de llamadas de la búsqueda desde {1,1}.*

3.6.3 Prueba de escritorio: ruta de menor costo

Paso	Posición	Símbolo	Costo de la casilla	Costo acumulado
0	{1,1}	P	— (no suma)	0
1	{1,2}	C	1	1
2	{1,3}	C	1	2
3	{1,4}	C	1	3
4	{1,5}	C	1	4
5	{2,5}	C	1	5
6	{3,5}	C	1	6
7	{3,4}	C	1	7
8	{4,4}	A	1	8
9	{5,4}	C	1	9
10	{5,5}	B	1	**10**

Tabla 17. *Acumulación del costo en la ruta de menor costo (10 movimientos, costo 10).*

3.6.4 Rutas encontradas

Resultados tomados de la ejecución real (figura de la sección 3.4.6):

Ruta	Movimientos	Costo	Recorrido
1	10	16	{1,1} -> {2,1} -> {3,1} -> {4,1} -> {5,1} -> {5,2} -> {5,3} -> {4,3} -> {4,4} -> {5,4} -> {5,5}
2	**8**	14	{1,1} -> {2,1} -> {3,1} -> {4,1} -> {5,1} -> {5,2} -> {5,3} -> {5,4} -> {5,5}
3	10	**10**	{1,1} -> {1,2} -> {1,3} -> {1,4} -> {1,5} -> {2,5} -> {3,5} -> {3,4} -> {4,4} -> {5,4} -> {5,5}
4	12	12	{1,1} -> {1,2} -> {1,3} -> {1,4} -> {1,5} -> {2,5} -> {3,5} -> {3,4} -> {4,4} -> {4,3} -> {5,3} -> {5,4} -> {5,5}

Tabla 18. *Rutas encontradas por Busqueda.rutas/3 con el mapa del enunciado.*

Coincide con el resumen del enunciado: **4 rutas**, la más corta de **8 movimientos** (costo 14) y la de menor costo con **costo 10 (10 movimientos)**. Como indica el enunciado, la ruta más corta no es la de menor costo, porque atraviesa terreno rocoso (R, costo 3) y difícil (D, costo 2).

3.7 Resultados del programa completo

Desde la carpeta `Problema2_Isla`:

```
elixir -r terreno.exs -r lector.exs -r validacion.exs -r mapa.exs -r ruta.exs -r
busqueda.exs -r reporte.exs -r salida.exs programa.exs
```

```
elixir -r terreno.exs -r lector.exs -r validacion.exs -r mapa.exs -r ruta.exs -r
busqueda.exs -r reporte.exs -r salida.exs programa.exs isla_invalida.txt
elixir -r terreno.exs -r lector.exs -r validacion.exs -r mapa.exs -r ruta.exs -r
busqueda.exs -r reporte.exs -r salida.exs programa.exs no_existe.txt
```

Figura 51. *Comandos de ejecución del problema 2: mapa del enunciado, mapa inválido y archivo inexistente.*

3.7.1 Mapa del enunciado

Contenido real de los archivos generados en `salidas/`:

```
Número total de rutas: 4
Ruta más corta: 8 movimientos
  {1,1} -> {2,1} -> {3,1} -> {4,1} -> {5,1} -> {5,2} -> {5,3} -> {5,4} -> {5,5}
Ruta de menor costo: costo 10 (10 movimientos)
  {1,1} -> {1,2} -> {1,3} -> {1,4} -> {1,5} -> {2,5} -> {3,5} -> {3,4} -> {4,4} -> {5,4}
-> {5,5}

Mapa con la ruta de menor costo marcada con *:
~ ~ ~ ~ ~ ~ ~
~ P * * * * ~
~ R T T T * ~
~ D C T * * ~
~ R T C * T ~
~ C D C * B ~
~ ~ ~ ~ ~ ~ ~
```

Figura 52. *Contenido real de resumen_isla.txt, con la ruta de menor costo marcada con * en el mapa.*

```
ruta;movimientos;costo;recorrido
1;10;16;{1,1} -> {2,1} -> {3,1} -> {4,1} -> {5,1} -> {5,2} -> {5,3} -> {4,3} -> {4,4} ->
{5,4} -> {5,5}
2;8;14;{1,1} -> {2,1} -> {3,1} -> {4,1} -> {5,1} -> {5,2} -> {5,3} -> {5,4} -> {5,5}
3;10;10;{1,1} -> {1,2} -> {1,3} -> {1,4} -> {1,5} -> {2,5} -> {3,5} -> {3,4} -> {4,4} ->
{5,4} -> {5,5}
4;12;12;{1,1} -> {1,2} -> {1,3} -> {1,4} -> {1,5} -> {2,5} -> {3,5} -> {3,4} -> {4,4} ->
{4,3} -> {5,3} -> {5,4} -> {5,5}
```

Figura 53. *Contenido real de rutas_isla.csv.*

Coincide con el resumen del enunciado: 4 rutas, la más corta de 8 movimientos y la de menor costo con costo 10 (10 movimientos).

```
a.exs -r ruta.exs -r busqueda.exs -r reporte.exs -r salida.exs programa.exs

----- validacion_isla.txt -----
El mapa es válido

----- rutas encontradas -----
Ruta 1:
{1,1} -> {2,1} -> {3,1} -> {4,1} -> {5,1} -> {5,2} -> {5,3} -> {4,3} -> {4,4} -> {5,4} -> {5,5}
Número de movimientos: 10
Costo: 16

Ruta 2:
{1,1} -> {2,1} -> {3,1} -> {4,1} -> {5,1} -> {5,2} -> {5,3} -> {5,4} -> {5,5}
Número de movimientos: 8
Costo: 14

Ruta 3:
{1,1} -> {1,2} -> {1,3} -> {1,4} -> {1,5} -> {2,5} -> {3,5} -> {3,4} -> {4,4} -> {5,4} -> {5,5}
Número de movimientos: 10
Costo: 10

Ruta 4:
{1,1} -> {1,2} -> {1,3} -> {1,4} -> {1,5} -> {2,5} -> {3,5} -> {3,4} -> {4,4} -> {4,3} -> {5,3} -> {5,4} -> {5,5}
Número de movimientos: 12
Costo: 12

Archivo generado: salidas/rutas_isla.csv

----- resumen_isla.txt -----
Número total de rutas: 4
Ruta más corta: 8 movimientos
  {1,1} -> {2,1} -> {3,1} -> {4,1} -> {5,1} -> {5,2} -> {5,3} -> {5,4} -> {5,5}
Ruta de menor costo: costo 10 (10 movimientos)
  {1,1} -> {1,2} -> {1,3} -> {1,4} -> {1,5} -> {2,5} -> {3,5} -> {3,4} -> {4,4} -> {5,4} -> {5,5}

Mapa con la ruta de menor costo marcada con *:
~ ~ ~ ~ ~ ~ ~
~ P * * * * ~
~ R T T T * ~
~ D C T * * ~
~ R T C * T ~
~ C D C * B ~
```

Figura 54. *Ejecución del programa con el mapa del enunciado: mapa válido, las 4 rutas, rutas_isla.csv generado y resumen con el mapa marcado.*

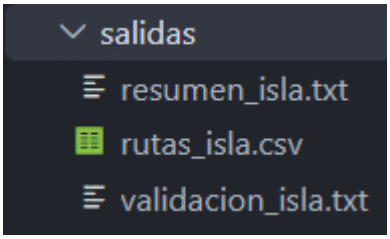


Figura 55. *Carpeta salidas/ del problema 2 con los tres archivos generados por el programa.*

3.7.2 Mapa inválido

```
Fila 3: tiene 6 posiciones; se esperaban 7
Posición {0,4}: el borde debe ser agua y contiene C
Posición {2,2}: el símbolo X no está en terrenos.csv
Hay 2 posiciones iniciales P: {1,1} y {4,3}
```

Figura 56. *Contenido real de validacion_isla.txt con isla_invalida.txt.*

El programa reporta los cuatro problemas y no ejecuta la búsqueda (figura de la sección 3.4.4). Cuando el mapa es válido, `validacion_isla.txt` contiene "El mapa es válido".

3.7.3 Archivo inexistente

Con `no_existe.txt` el programa muestra `El archivo .../entradas/no_existe.txt no existe` y termina sin excepción (figura de la sección 3.4.4).

3.8 Casos de error

Caso	Entrada	Resultado	Estado
Campos faltantes en terrenos	`Terreno.parse("C,camino")`	`{:error, "campos faltantes"}`	Verificado
Transitable inválido	`Terreno.parse("C,camino,quizas,1")`	`{:error, "transitable debe ser si o no"}`	Verificado
Costo no numérico	`Terreno.parse("C,camino,si,abc")`	`{:error, "costo no numérico"}`	Verificado
Archivo inexistente (lector)	`Lector.leer_isla("entradas/no_existe.txt")`	Mensaje claro, sin excepción	Verificado
Mapa inválido	isla_invalida.txt	4 problemas, sin buscar rutas	**Verificado**
Archivo inexistente (programa)	Renombrar un archivo de entrada	Mensaje claro, sin excepción	**Verificado**

Tabla 19. *Casos de error del problema 2.*

3.9 Estado de validación del problema 2

Requisito del enunciado	Estado
Búsqueda recursiva de todas las rutas, solo en 4 direcciones y sin repetir posiciones	Verificado
Movimientos y costo de cada ruta; costo según terrenos.csv, sin contar la posición inicial	Verificado
Símbolos y costos tomados de terrenos.csv, no escritos en el código	Verificado
Structs %Terreno{} y %Ruta{}; movimientos calculados a partir de las posiciones	Verificado
Descarte de posiciones no transitables con pattern matching sobre el struct	Verificado
Validación recursiva del mapa (4 reglas) sin Enum ni Stream	Verificado
Lectura con File.stream!, ignorando vacías y comentarios	Verificado
Total de rutas, ruta más corta y ruta de menor costo	Verificado
rutas_isla.csv, resumen_isla.txt con mapa marcado con *, validacion_isla.txt	**Verificado**
Escritura línea por línea y archivo inexistente a nivel de programa	**Verificado**

Tabla 20. *Estado de los requisitos del problema 2.*

3.10 Bitácora de uso de IA (problema 2)

Registro de las interacciones con la herramienta de IA durante el desarrollo de este problema.

3.10.1 Interacciones registradas

Módulo	Qué se pidió a la IA	Qué se hizo con la respuesta
Organización	Comenzar el problema 2	La IA creó la carpeta, los archivos vacíos de los módulos y los archivos isla.txt y terrenos.csv con los datos del enunciado, y calculó de antemano con un script auxiliar las 4 rutas esperadas para poder comparar los resultados

Módulo	Qué se pidió a la IA	Qué se hizo con la respuesta
`terreno.exs`	Plantilla del struct y de `parse/1` con `with`	Completé todos los espacios correctamente en el primer intento
`ruta.exs`	Más pistas sobre el separador del CSV	Completé los separadores (`" -> "` y `;`); el espacio de `length(posiciones) - 1` quedó vacío y lo completó la IA
`lector.exs`	Lector de terrenos e isla	Lo escribió la IA reutilizando el del problema 1, por tiempo; yo lo probé con los dos archivos y con un archivo inexistente
`validacion.exs`	Plantilla con 5 espacios y explicación de las pistas 1, 4 y 5	Completé el caso base `[]`, la variable repetida y `[_una]`; en las llamadas recursivas dejé `f` y `c` sin sumar 1 y la IA lo corrigió explicando que las coordenadas no avanzarían. La IA creó isla_invalida.txt para las pruebas
`mapa.exs`	Plantilla con los 3 vecinos por completar	Completé las tres direcciones correctamente
`busqueda.exs`	Plantilla con 5 espacios y explicación de las pistas 4 y 5	El costo inicial quedó bien; puse `posicion` en lugar de `resto` y la IA lo corrigió. Al guardar desde VS Code se sobrescribió la corrección, Elixir mostró el aviso "variable resto is unused" y la IA lo corrigió de nuevo. `nil`, `false` y `costo_casilla` quedaron correctos
Integración	Reporte, salida y programa principal	Los escribió la IA por tiempo; yo ejecuté el programa con el mapa del enunciado, el mapa inválido y un archivo inexistente, y los resultados coincidieron con lo esperado
Pruebas	Comandos para probar cada módulo	Ejecuté las pruebas y tomé las capturas; en la validación olvidé leer el mapa inválido antes de usarlo (error "undefined variable mala")
Documentación	Documentar el problema 2 a partir de las 10 capturas	La IA organizó las capturas y redactó este capítulo; yo debo revisarlo

Tabla 21. *Bitácora de uso de IA en el problema 2.*

3.10.2 Un ejemplo concreto

La interacción que más me enseñó en este problema fue el aviso de Elixir al cargar `busqueda.exs`: "variable resto is unused". En la llamada recursiva de `explorar` yo había escrito `explorar(posicion, ...)` en lugar de `explorar(resto, ...)`. La IA ya lo había corregido, pero al guardar el archivo desde VS Code volví a escribir encima mi versión con el error. Al cargar el archivo, Elixir mostró el aviso y la IA me explicó que significaba que había sacado `resto` de la lista pero nunca lo usaba, y que eso pasaba justamente porque la llamada recursiva no recibía la lista de vecinos que faltaban.

Aprendí dos cosas. Primero, que un warning no detiene el programa pero casi siempre señala un error de lógica: con `posicion` la función no habría encontrado ninguna cláusula de `explorar` que encajara. Segundo, que en una recursión sobre listas la llamada siempre debe recibir el `resto`, para que el problema se haga más pequeño y llegue al caso base. Era el mismo error que había cometido en la depuración del problema 1 con `buscar_par(previa, ...)`.

4. Reflexión sobre el uso de inteligencia artificial

Esta sección responde las preguntas del enunciado sobre el uso de inteligencia artificial en todo el parcial. Para redactarla me apoyé en la IA a partir del registro de la sesión de trabajo; revisé cada afirmación y corresponde a lo que ocurrió. Las bitácoras detalladas de cada problema están al final de su capítulo (sección 2.11 para el problema 1).

4.1 Herramientas y agentes utilizados

Herramienta	Uso en el parcial
Claude (modo Cowork, con acceso al navegador y a la carpeta del proyecto)	Exploración del material del curso en Classroom y en las guías del docente; diseño de la arquitectura; explicaciones de conceptos; plantillas de código con espacios para completar; revisión y corrección de los archivos; escritura de los módulos de integración
Visual Studio Code y terminal (iex / elixir)	Escritura del código y ejecución de todas las pruebas

Tabla 22. *Herramientas utilizadas.*

4.2 Propuestas distintas a lo visto en clase

La IA mencionó `MapSet` como la estructura ideal para representar conjuntos de ciudades en el Stream de alcance, pero no se usó porque no hace parte de las guías del curso; en su lugar se emplearon listas sin repetidos y ordenadas (`Enum.uniq` y `Enum.sort`). También se usó `Stream.chunk_every(2, 1)` para comparar cada paso con el siguiente, una forma de aplicar funciones vistas en clase a un problema nuevo.

Un `MapSet` no permite elementos repetidos, pertenecer o no al conjunto se consulta sin recorrer todos los elementos, y dos conjuntos con las mismas ciudades son iguales sin importar el orden. Para el Stream de alcance habría sido la estructura más natural. Decidí no usarlo porque no está en las guías del curso y el parcial pide resolver los problemas con lo visto en clase.

Lo que aprendí al usar listas es que `Enum.sort` no es solo para que la salida se vea ordenada: sin ordenar, dos pasos con las mismas ciudades en distinto orden se verían diferentes y `Stream.take_while` nunca detectaría que la secuencia se estabilizó. También aprendí a usar `Stream.chunk_every(2, 1)` para comparar cada paso con el siguiente, y que un error pequeño como escribir `=!` en lugar de `!=` no produce un error de compilación sino un resultado equivocado en silencio, porque Elixir lo interpreta como una asignación seguida de una negación.

4.3 Conclusiones

Lo que entendí por mí mismo. Puedo explicar por qué `Conexion` es un solo struct con tres campos y para qué sirve `@enforce_keys`; cómo `parse/1` limpia, separa y valida una línea, y por qué solo se acepta el resultado `{numero, ""}` de `Integer.parse`; cómo se arma el pipeline con Streams y que nada se lee hasta que `Enum.reduce` consume el stream; y la idea general de la búsqueda: probar cada vecino, no entrar a una ciudad que ya está en el camino y volver atrás para probar el siguiente.

Lo que todavía depende de la IA. Escribir funciones recursivas desde cero. Completé las plantillas de la depuración, el alcance y la búsqueda, pero me equivoqué varias veces en el caso base o en el valor que se pasa a la siguiente llamada (por ejemplo, `previa` en lugar de `resto`, o `distancia + camino` en lugar de `distancia + km`). También dependo de la IA en los módulos de integración (`reporte.exs`, `salida.exs` y `programa.exs`), que fueron escritos por ella por el tiempo; antes de la sustentación debo estudiarlos hasta poder explicarlos y modificarlos.

Dificultades que no eran de programación. Gran parte del tiempo se fue en el entorno: en PowerShell `iex` no abre Elixir y hay que usar `iex.bat`; ejecutar los comandos desde la carpeta equivocada; archivos que creía guardados en VS Code y estaban vacíos en el disco; y variables que se pierden al cerrar iex.

Qué haría distinto. Empezar antes, porque trabajé de forma individual y con poco tiempo; activar el guardado automático de VS Code desde el inicio; hacer la prueba de escritorio a mano antes de completar cada función recursiva; escribir mensajes de commit descriptivos desde el primer commit; e intentar escribir yo mismo al menos uno de los módulos de integración, para depender menos de la IA en esa parte.

Anexo A. Control de evidencias

#	Evidencia	Sección	Estado
1	Pruebas de `Conexion.parse/1` y `@enforce_keys` en iex	2.4.1	Hecho
2	`Lector.leer_conexiones/1` con archivo existente e inexistente	2.4.2	Hecho
3	Depuración y grafo en iex	2.4.4	Hecho
4	Pruebas de `Ruta`	2.4.6	Hecho
5	`Alcance.secuencia(...) |> Enum.take(6)`	2.6.3	Hecho
6	Ejecución completa del programa	2.8	Hecho
7	Carpeta salidas/ con los 4 archivos	2.8	Hecho
8	Archivo de entrada inexistente	2.9	Hecho
9	Redacción de la reflexión sobre IA	2.11 y 4	Hecho
10	Problema 2: validación de isla_invalida.txt	3.4.4	Hecho
11	Problema 2: capturas de la ejecución con el mapa del enunciado y de salidas/	3.7.1	Hecho
12	Problema 2: revisión de la bitácora de IA	3.10.2	Hecho

Tabla 23. *Control de evidencias del documento.*